

MySQL 面试题速记通关版 _ 面试刷题 mianshiya.com

本资源来自面试鸭: <https://www.mianshiya.com>

推荐更多免费学编程资源:

1. 编程导航学习网站: [学编程、做项目、拿 Offer!](#)
2. 企业高频面试题库: [开始刷题, 面试遇原题!](#)
3. 精选简历模板大全: [1 分钟搞定简历!](#)
4. AI 资源导航网站: [获取最新 AI 黑科技!](#)
5. 1 对 1 模拟面试: [随时随地提升面试能力](#)

MySQL 的 Doublewrite Buffer 是什么? 它有什么作用?

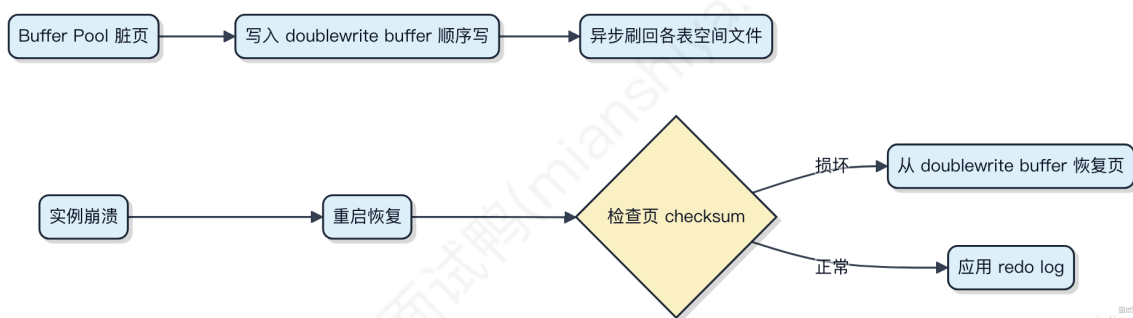
MySQL 的 Doublewrite Buffer 是个防止数据页写入过程中出现“部分写”问题的机制。InnoDB 每次刷脏页时, 并不会直接把页写到表空间文件, 而是先写一份到共享表空间里的连续区域——也就是 doublewrite buffer, 再从这里分批写回真正的数据文件。

这个机制的核心价值在于解决崩溃恢复时的数据页不一致。磁盘写入以 4K 或 8K 为单位, 而 InnoDB 页通常是 16K。如果在写一个 16K 页的过程中断电, 可能只写了前 8K, 导致页处于“撕裂”状态。重启后没法靠 redo log 恢复这种损坏页, 因为 redo 记录的是逻辑变更, 前提得有个完整的页。

有了 doublewrite buffer, 恢复时会做两件事: 1) 检查数据页自带的 checksum, 判断是否完整 2) 如果不完整, 就去 doublewrite buffer 里找这个页的副本, 用它来恢复

虽然听起来像“双写”会带来性能开销, 但其实写 doublewrite buffer 是顺序写, 批量进行, 对性能影响不大。反而是在没有它的情况下, 为了保证页完整性, 不得不开启 `innodb_checksums` 并配合每页重做, 代价更高。

典型场景比如大事务提交后大量脏页刷新, 或者缓冲池压力大频繁刷脏页时, doublewrite buffer 就在后台默默扛住一致性风险。



注意: 某些文件系统如 ZFS 或使用 DAX 模式的持久化内存可以禁用该功能, 因为它们本身已提供原子写。

MySQL 默认的事务隔离级别是什么? 为什么选择这个级别?

MySQL 默认的事务隔离级别是**可重复读** (Repeatable Read)。这个级别能解决脏读和不可重复读问题, 大多数业务场景下够用又不会太影响并发性能。

- 1) 脏读是指一个事务读到了另一个未提交事务的数据, 这在任何生产环境都是不能接受的。
- 2) 不可重复读是指同一个事务内两次查询结果不一致, 因为期间其他事务修改了数据。**可重复读**通过 MVCC (多版本

并发控制) 机制让事务看到一致的快照, 避免这个问题。

但它不解决幻读问题——不过 MySQL 的 InnoDB 在这个级别用了间隙锁 (Gap Lock) + 行锁的组合, 实际上把幻读也基本遏制住了。比起串行化那种一刀切的锁表操作, 它在数据一致性与并发之间取得了不错的平衡。

代码上你不需要显式设置, 除非要改:

```
SELECT @@transaction_isolation;  
-- 默认输出 REPEATABLE-READ
```

如果系统对一致性要求极高, 比如金融核心账务, 可以升到串行化。但大部分 Web 应用, 像电商库存、用户积分这类操作, **可重复读**完全扛得住, 还不会因为过度加锁导致线程堵死。

要不要升级隔离级别, 得看业务能不能容忍一点点幻读风险。其实很多项目连这个级别的坑都没遇到过, 就是因为底层存储引擎做了额外保护。

什么是数据库的逻辑删除? 数据库的物理删除和逻辑删除有什么区别?

数据库里删数据, 不是所有场景都直接 DELETE 就完事。有时候得留个痕迹, 这时候就用到逻辑删除。

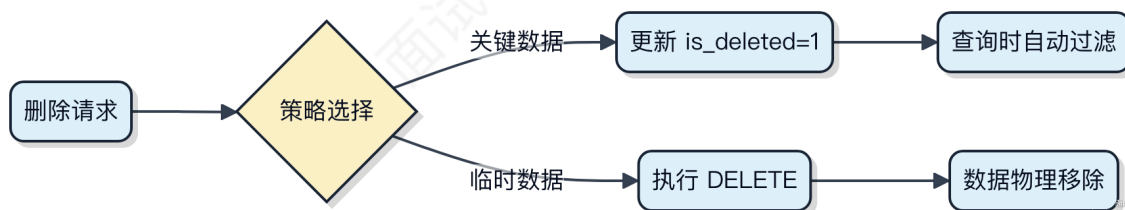
逻辑删除本质是**更新操作**。给表加个字段, 比如 `is_deleted` 或 `del_flag`, 删数据时把这字段从 0 改成 1, 数据本身还留在表里。查的时候约定好, 默认只查 `is_deleted = 0` 的记录。像 MyBatis-Plus 这种框架, 直接内置了自动过滤逻辑删除数据的能力, 开发时几乎感知不到。

物理删除就是真正的 DELETE 语句, 数据从存储引擎层被移除, 行记录直接消失, 空间后续可以复用。

两者的差别主要在几个地方:

- 1) 数据可恢复性: 逻辑删除能回溯, 误删也能捞回来, 审计、合规场景特别需要。物理删除除非靠 binlog 恢复, 不然基本没戏。
- 2) 性能影响: 大批量逻辑删除会导致表数据膨胀, 查询变慢, 尤其是没有有效索引时。物理删除释放空间更彻底, 但高并发下可能引发锁竞争和主从延迟。
- 3) 关联数据处理: 物理删除常要处理外键约束, 可能级联删一堆表; 逻辑删除通常只要改状态, 脏活累活少很多。
- 4) 存储成本: 逻辑删除长期不清理, 磁盘压力会越来越大, 得配合归档机制。

实际项目里, 用户账户、订单这种关键数据, 基本都用逻辑删除。日志类、流水类数据, 过期就删, 适合物理删除或定时归档。



MySQL 中 int(11) 的 11 表示什么?

MySQL 里 `int(11)` 的 11 压根不是指存储长度或取值范围, `int` 固定占 4 个字节, 最大存 21 亿左右。

这个 11 其实是显示宽度，只在特定场景下起作用，比如配合 `ZEROFILL` 时，不足位数会用 0 填充。例如定义为 `int(11) ZEROFILL`，存个 1，查出来就是 00000000001。

但现在这特性基本没人用，`ZEROFILL` 还有性能隐患，官方从 8.0.17 版本开始已经废弃了这一语义。所以现在写 `int(11)` 和 `int(5)` 在实际运行中完全没区别。

建表的时候如果还纠结括号里的数字，纯属浪费时间。真正要注意的是数据类型本身，比如别把用户 ID 用 `int` 存，万一哪天超 21 亿就搞不定了，直接上 `bigint` 才省心。

```
-- 下面这两行，效果完全一样
id int(11) NOT NULL
id int(5) NOT NULL
```

开发里看到 `int(11)` 别慌，99% 的情况它只是历史遗留的书写习惯，不影响任何行为。

MySQL 中 varchar 和 char 有什么区别？

`varchar` 和 `char` 都是用来存字符串的，但用法和底层处理差别挺大。

`char` 是定长的，你定义成 `char(10)`，那它就占 10 个字符的空间，不管实际存的是 "a" 还是 "hello"，都会补空格到 10。查询快，适合存长度固定的值，比如性别、状态码这种 1-2 位的标识。但浪费空间，尤其是字段定义得大而实际内容短的时候。

`varchar` 是变长的，存多少占多少，最大支持 65535 字节（受限于行大小）。它需要额外 1-2 个字节记录长度，所以读写时多一点开销。适合长度不一的场景，比如用户名、地址这些。但因为长度可变，在行内存储结构上更灵活，空间利用率高。

1) `char` 全部按固定长度分配存储，尾部空格会被截断（除非启用 `PAD_CHAR_TO_FULL_LENGTH` 模式） 2) `varchar` 实际占用空间 = 内容长度 + 长度标识字节（内容小于 255 字节用 1 字节，否则 2 字节） 3) 两者在检索时，`char` 的性能略好，因为位置固定；`varchar` 要先读长度再定位内容

举个例子：

```
CREATE TABLE t (a char(10), b varchar(10));
INSERT INTO t VALUES ('abc', 'abc');
```

这行数据中，a 列实际占 10 字符，b 列只占 3 字符 + 1 字节长度信息。

注意行大小限制，一张表所有列加起来不能超过约 65KB，所以别随便把大文本塞进 `char` 或超长 `varchar`。

MySQL 中的数据排序是怎么实现的？

MySQL 的排序其实有两种路径，走不走索引决定了性能差很多。

如果 `order by` 的字段有合适的索引，直接利用索引的有序性输出就行。索引本身就是排好序的 B+ 树，查出来天然有序，压根不需要额外排序。这种是理想情况，比如主键或联合索引的最左前缀匹配。

但大多数时候还是得排序，这时候就进入 **sort buffer** 流程。MySQL 会从引擎层读取满足条件的行，把需要的列（`select` 的字段或用于排序的字段）放进一块内存区域叫 `sort buffer`。当这块内存不够时，比如要排上百万条数据，就会分段排序，每段单独快排，写到临时文件，最后做归并排序。这个过程叫 **filesort**，虽然名字带 `file`，但其实是磁盘和内存配合的操作。

有个关键参数是 `sort_buffer_size`，调大能减少磁盘临时文件的使用，但每个连接独占这份内存，设太大容易吃光内存。

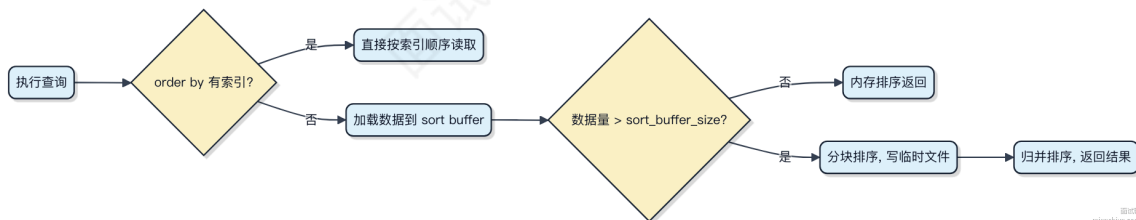
举个例子，你查订单列表按创建时间排序，又没加索引，那每来一个请求都得进 `filesort`，高并发下 CPU 和 IO 直接拉满。所以慢查询里经常看到 Using `filesort` 就得警惕了。

代码层面没法直接控制排序算法，但可以通过索引设计规避。比如：

```
SELECT name, create_time FROM orders ORDER BY create_time;
```

给 `create_time` 加索引就能跳过 `filesort`。

- 1) 索引有序直接返回
- 2) 无索引走 `sort buffer` + 可能的磁盘临时文件
- 3) `max_length_for_sort_data` 控制用单路还是双路排序，老版本有回表问题



MySQL 中 EXISTS 和 IN 的区别是什么？

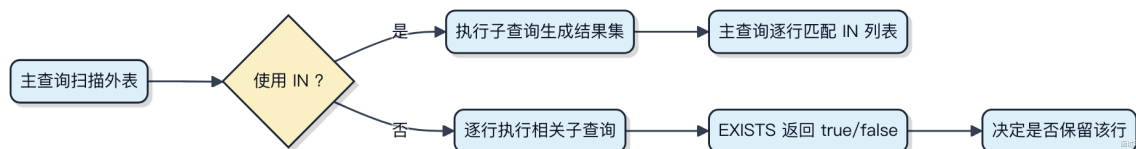
EXISTS 和 IN 都用来处理子查询，但底层玩法完全不同。

1) IN 是把子查询的结果全捞出来，存成一个临时的值列表，然后拿主查询的每一行去这个列表里匹配。比如 `SELECT * FROM user WHERE dept_id IN (SELECT id FROM dept)`，它会先把所有 `dept.id` 查出来，再逐个比对。如果子查询结果很大，比如上万条，这个临时列表就很占内存，性能容易崩。

2) EXISTS 不关心具体值，只判断是否存在。它采用**相关子查询**的方式，对外表每一条记录执行一次子查询。只要子查询能返回一行，就认为存在。比如 `SELECT * FROM user u WHERE EXISTS (SELECT 1 FROM dept d WHERE d.id = u.dept_id)`，这里子查询依赖外层 `user` 表的字段，相当于“为用户查一下有没有对应的部门”。一旦命中就停止，效率更高。

一般情况下，**外表小、内表大**用 EXISTS 更合适，因为驱动行数少，关联查询快；**外表大、内表小**用 IN 更好，因为小结果集做哈希查找代价低。

MySQL 8.0 对两者都有优化，但在复杂场景下手动选对依然关键。比如在分页查询中嵌套条件时，EXISTS 往往能避免全表扫描。



什么是分库分表？分库分表有哪些类型（或策略）？

分库分表本质是为了解决单库扛不住数据量和并发压力的问题。当一张表数据达到千万甚至上亿级别，查询变慢、锁竞争激烈，主从复制延迟也会加剧。这时候就得把数据拆开，让多个数据库一起干活。

1) 垂直分库

按业务模块拆分，比如用户服务用一个库，订单服务用另一个库。这种拆法最常见于微服务架构中，像电商系统里把商品、库存、交易拆到不同库，彼此独立部署，互不影响。

2) 垂直分表

把一张宽表按字段拆开，比如用户表的登录信息和扩展资料分开存储。一般用在表字段特别多、冷热分明的场景，减少 I/O 开销。

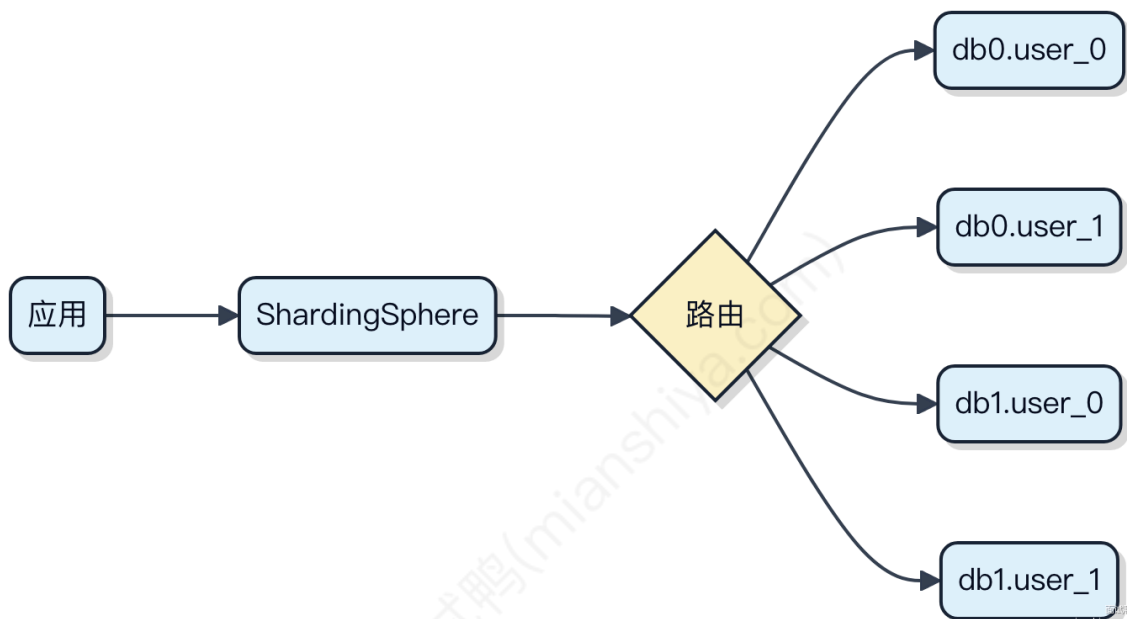
3) 水平分表

同一张表的数据按某种规则打散到多个结构相同的表里，比如用户表按 user_id 取模分成 16 张表。适合写多读少、数据增长快的场景，能有效降低单表容量。

4) 水平分库

在水平分表基础上再跨数据库实例存储，比如把订单数据按 order_id 分到 8 个 MySQL 实例上。这是真正意义上的分布式拆分，能大幅提升存储和吞吐能力。

分片键 (Sharding Key) 的选择特别关键，它决定了数据是否均匀、查询能否下推。常见的有取模、范围、一致性哈希等策略。像 ShardingSphere、MyCat 这类中间件就是靠解析 SQL + 路由规则自动完成数据寻址的。



需要注意的是，一旦分库分表，跨库事务、全局排序、聚合查询就变得复杂，得靠柔性事务或引入 ES 做汇总查询来兜底。

MySQL 中 VARCHAR(100) 和 VARCHAR(10) 的区别是什么？

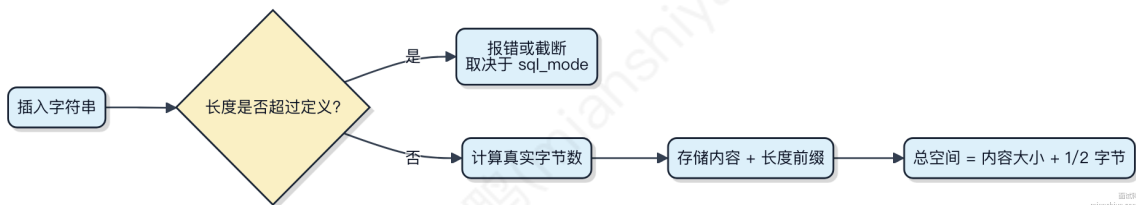
字段定义里的数字是字符长度限制，不是字节占用。VARCHAR(10) 最多存 10 个字符，VARCHAR(100) 最多存 100 个字符，超长会截断或报错，取决于 SQL 模式。

实际存储空间上，两者都只用够用的空间。比如都存 "abc"，在 UTF8MB3 编码下都是 3 字节内容 + 1 或 2 字节长度前缀。长度前缀根据最大定义长度决定：小于等于 255 用 1 字节，大于 255 用 2 字节。所以 VARCHAR(10) 和 VARCHAR(100) 都用 1 字节前缀，**存储开销一样**。

但别乱设大值。虽然 MySQL 层面不浪费空间，但会影响内存分配和临时表行为。比如排序时，MySQL 会按最大长度预估内存，`VARCHAR(100)` 可能比 `VARCHAR(10)` 多占 10 倍内存，容易把数据挤到磁盘临时表，性能就崩了。

另外 ORM 映射、应用层校验也会依赖这个长度。设成 `VARCHAR(10000)` 虽然技术上可行，但会让其他开发者困惑，也可能触发一些框架的低效路径。

所以建议按实际需要定长度，给点余量就行，比如用户名用 `VARCHAR(64)`，短标题用 `VARCHAR(255)`。别图省事全用 `VARCHAR(255)` 或直接拉满。



MySQL 中 `count(*)`、`count(1)` 和 `count(字段名)` 有什么区别？

`count(*)`、`count(1)` 和 `count(字段名)` 看起来都是统计行数，但底层处理方式其实有差别，用错了可能影响性能甚至结果。

`count(*)` 是标准的行计数方式，它不关心具体列，只统计**所有行**，包括 `null` 值。MySQL 优化器对它做了专门优化，尤其是 InnoDB 引擎下会选择成本最低的索引树来遍历，不一定真的全表扫描。

`count(1)` 其实和 `count(*)` 非常接近，因为 `1` 是常量，每一行都能取到值，所以也是统计所有行。在大多数情况下，它的执行计划和 `count(*)` 一样，性能几乎无差别。别被“1”误导，它不是按第一列算。

真正有区别的是一些人写的 `count(字段名)`。这个函数只统计该字段**非空 (not null)** 的行。如果这个字段允许为 `null`，那结果就可能比 `count(*)` 少。比如 `count(email)` 会漏掉 email 为空的用户，这在统计总数时容易出错。

- 1) `count(*)` 统计所有行，推荐用于总行数统计
- 2) `count(1)` 效果基本等同于 `count(*)`，可互换
- 3) `count(字段名)` 只统计该字段非空行，要小心语义差异

一般情况下，想查总行数就老老实实用 `count(*)`，别搞花活。有些 ORM 框架默认生成 `count(1)`，问题不大，但看到 `count(id)` 还是得确认下 `id` 是不是主键且非空。

MySQL 中的 Log Buffer 是什么？它有什么作用？

MySQL 的 InnoDB 存储引擎在处理事务时，不会每次写操作都直接刷到磁盘的 redo log 文件，那样性能会很差。它先写到一块内存区域，这块区域就是 **Log Buffer**。

- 1) 当执行 INSERT、UPDATE、DELETE 这类事务操作时，InnoDB 会生成对应的 redo 日志，这些日志优先写入 Log Buffer。
- 2) Log Buffer 在内存中，写入速度远快于磁盘 IO，能极大减少事务提交时的等待时间。
- 3) 后续由后台线程根据策略（比如每秒一次、事务提交触发等）将缓冲中的日志批量刷到磁盘的 redo log 文件，确保崩溃恢复时数据不丢。

这个过程其实和 write-ahead logging (WAL) 机制紧密相关，redo log 必须先于对应的数据页落盘。Log Buffer 就是 WAL 中的关键缓存层。

你可以通过 `innodb_log_buffer_size` 参数调整它的大小，一般默认 16MB，对于大量大事务或高并发 insert 场景，可以调大到 64MB 或更高，避免频繁刷盘。



压根不经过 Log Buffer 直接写磁盘，系统基本就扛不住写压力了。脏活累活它承担了一大半。

在 MySQL 中存储金额数据，应该使用什么数据类型？

金额这种数据，最怕的就是精度丢失，尤其是涉及到加减乘除的时候。你要是用 float 或 double，看着能存，但算着算着就差了几分钱，线上一跑账对不上，背锅是迟早的事。

MySQL 里处理金额，得用 **DECIMAL**。它不是近似值，是精确的，存多少就是多少。比如 `DECIMAL(10,2)`，表示总共最多 10 位数字，小数点后占 2 位，能存像 99999999.99 这样的值，够大多数业务用了。你要是做金融系统，可能得 `DECIMAL(18,4)` 或更高，看实际需求。

```
CREATE TABLE orders (  
  id BIGINT PRIMARY KEY,  
  amount DECIMAL(10,2) NOT NULL COMMENT '订单金额，单位：元'  
);
```

有人说用 int 存“分”也行，比如 100 元存成 10000 分。这确实能避免小数，但你要在代码里反复转换单位，逻辑容易出错，读数据库也不直观。除非你整个系统都统一这么干，不然没必要。

总之，别碰 float/double。该用 **DECIMAL** 的地方就得用它，简单直接，不给问题留机会。

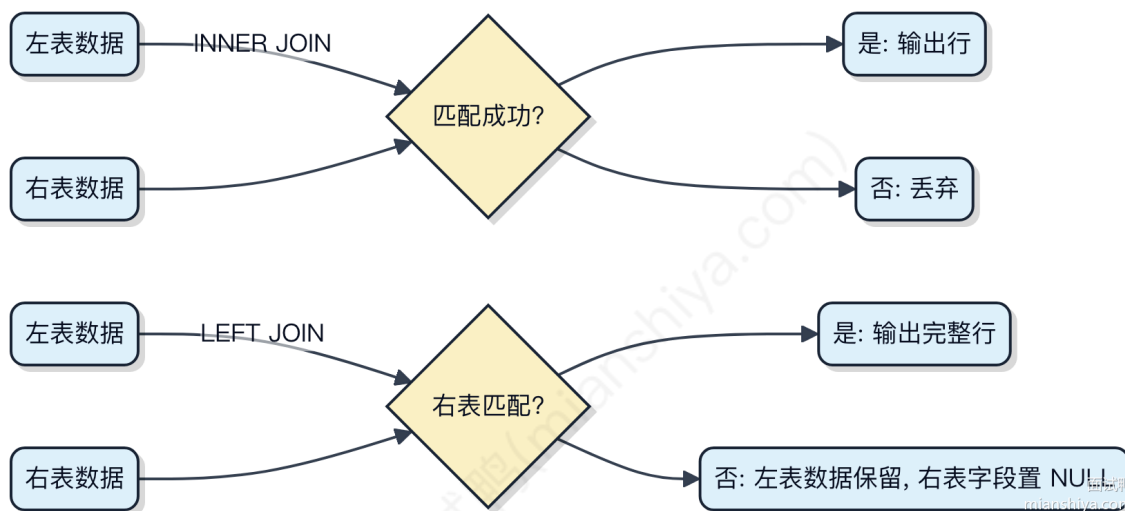
MySQL 中 INNER JOIN、LEFT JOIN 和 RIGHT JOIN 的区别是什么？

内连接只返回两边都能匹配上的行，左连接会保留左边表的全部数据，右边没有匹配的就补 NULL，右连接反过来。

- 1) INNER JOIN 是最严格的，驱动表和被驱动表做关联时，必须满足 ON 条件才会出现在结果里。比如查订单和用户信息，只有用户存在且订单归属该用户才输出。
- 2) LEFT JOIN 保证左表的每一行都出现，即使右表没找到匹配项。常见于统计场景，像列出所有商品，不管有没有销量记录，都要展示出来，用 `SUM(ifnull(sales,0))` 累加。
- 3) RIGHT JOIN 和 LEFT JOIN 是镜像关系，实际很少用，一般统一写成 LEFT JOIN 避免混淆。

```
-- 内连接: 仅返回 user 和 order 能关联上的数据  
SELECT u.name, o.amount  
FROM user u INNER JOIN order o ON u.id = o.user_id;  
  
-- 左连接: 返回所有用户，没订单的 amount 为 NULL  
SELECT u.name, o.amount  
FROM user u LEFT JOIN order o ON u.id = o.user_id;
```

注意点：WHERE 和 ON 的执行顺序不同，ON 先过滤用于关联的数据，而 WHERE 是最终结果的筛选。LEFT JOIN 中如果把右表字段放 WHERE 里可能意外过滤掉 NULL 行，导致变成事实上的内连接。



MySQL 中 `LIMIT 100000000, 10` 和 `LIMIT 10` 的执行速度是否相同?

MySQL 里 `LIMIT 100000000, 10` 和 `LIMIT 10` 执行速度完全不同, 前者慢得多。

原因在于 `LIMIT offset, size` 的执行机制。MySQL 会先扫描前 `offset + size` 行, 然后丢弃前 `offset` 行, 返回后面的 `size` 行。也就是说, `LIMIT 100000000, 10` 要真实地跳过一个亿行数据, 哪怕这些数据最终不用返回。这个过程是逐行检索的, 尤其是当没有覆盖索引时, 每跳过一行都可能涉及主键回表, 代价非常高。

而 `LIMIT 10` 只需找到前 10 条记录就结束, 扫描量极小, 自然快得多。

这种问题在分页查询中特别常见。比如用 `OFFSET` 做深度分页, 到后面页面时响应明显变慢。解决方案一般有两种:

1) 使用 **游标分页** (Cursor-based Pagination), 比如基于上一页最后一条记录的主键或唯一排序字段继续查下一页:

```
SELECT * FROM table WHERE id > last_seen_id ORDER BY id LIMIT 10;
```

2) 如果必须用 `offset`, 可以先通过索引覆盖查出主键, 再关联原表, 减少回表开销:

```
SELECT t.* FROM table t
INNER JOIN (SELECT id FROM table ORDER BY id LIMIT 100000000, 10) AS tmp ON t.id =
tmp.id;
```

真正解决这类性能问题的关键是避免全表扫描和大量数据跳过, **索引下推**和**最小化数据访问**才是根本思路。

MySQL 中 `DATETIME` 和 `TIMESTAMP` 类型的区别是什么?

`DATETIME` 和 `TIMESTAMP` 都用来存时间, 但底层设计目标不一样, 用错场景容易踩坑。

1) 存储范围不同

`DATETIME` 能存从 1000-01-01 到 9999-12-31 的时间, 精度到秒。`TIMESTAMP` 只支持 1970-01-01 到 2038-01-19, 受 32 位时间戳限制, 过了 2038 年会溢出。

2) 时区处理差异大

TIMESTAMP 实际存的是 UTC 时间戳，读写时根据当前会话的 `time_zone` 自动转换。插入时转成 UTC 存，查询时按本地时区展示。DATETIME 不带时区信息，存什么就是什么，不转换。

3) 存储空间也不同

DATETIME 占 8 字节，TIMESTAMP 只占 4 字节。如果数据量上亿，TIMESTAMP 能省不少空间。

4) 自动初始化和更新行为

两者都支持 `DEFAULT CURRENT_TIMESTAMP` 和 `ON UPDATE CURRENT_TIMESTAMP`，但 TIMESTAMP 在旧版本中更早支持这些特性，某些情况下一个表只能有一个 TIMESTAMP 自动更新。

```
CREATE TABLE events (  
  id INT PRIMARY KEY,  
  dt DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
  ts TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP  
);
```

一般建议：需要跨时区保存时间（比如日志时间、交易发生时间），选 **TIMESTAMP**；单纯记录某个时刻且不想被时区干扰，比如“预约时间”，就用 DATETIME。别让 2038 年问题搞不定未来系统。

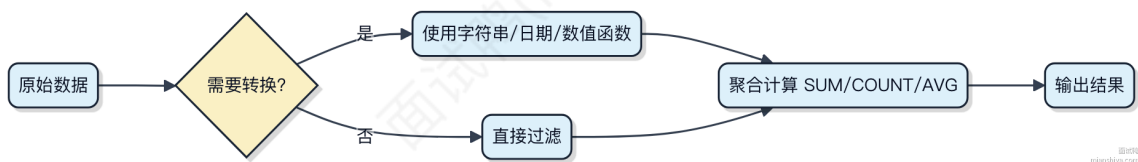
在 MySQL 中，你使用过哪些函数？

MySQL 里的函数用得多了，关键看场景。字符串处理像 `CONCAT` 拼接、`SUBSTRING` 截取，日志分析或者生成报表时特别常见。日期函数几乎天天见，比如 `DATE_ADD`、`DATEDIFF`，做订单生命周期统计或者用户活跃度分析都绕不开。

数值计算这块，`ROUND` 控制小数位，`IFNULL` 处理空值很实用，避免后续计算出错。聚合函数像 `SUM`、`COUNT`、`AVG` 更是家常便饭，配合 `GROUP BY` 出数据快。有时候还得写 `CASE WHEN` 做条件判断，比如把订单状态码转成可读文本。

但要注意，尽量别在 `WHERE` 条件里对字段套函数，比如 `WHERE YEAR(create_time) = 2024`，这会让索引失效，全表扫描，性能直接崩。应该写成 `WHERE create_time >= '2024-01-01' AND create_time < '2025-01-01'`，让查询能走索引。

复杂逻辑建议放在应用层，或者用视图封装，别一股脑塞给数据库算。毕竟数据库的 CPU 成本高，还可能影响主从复制延迟，尤其是大表上跑一堆函数表达式。



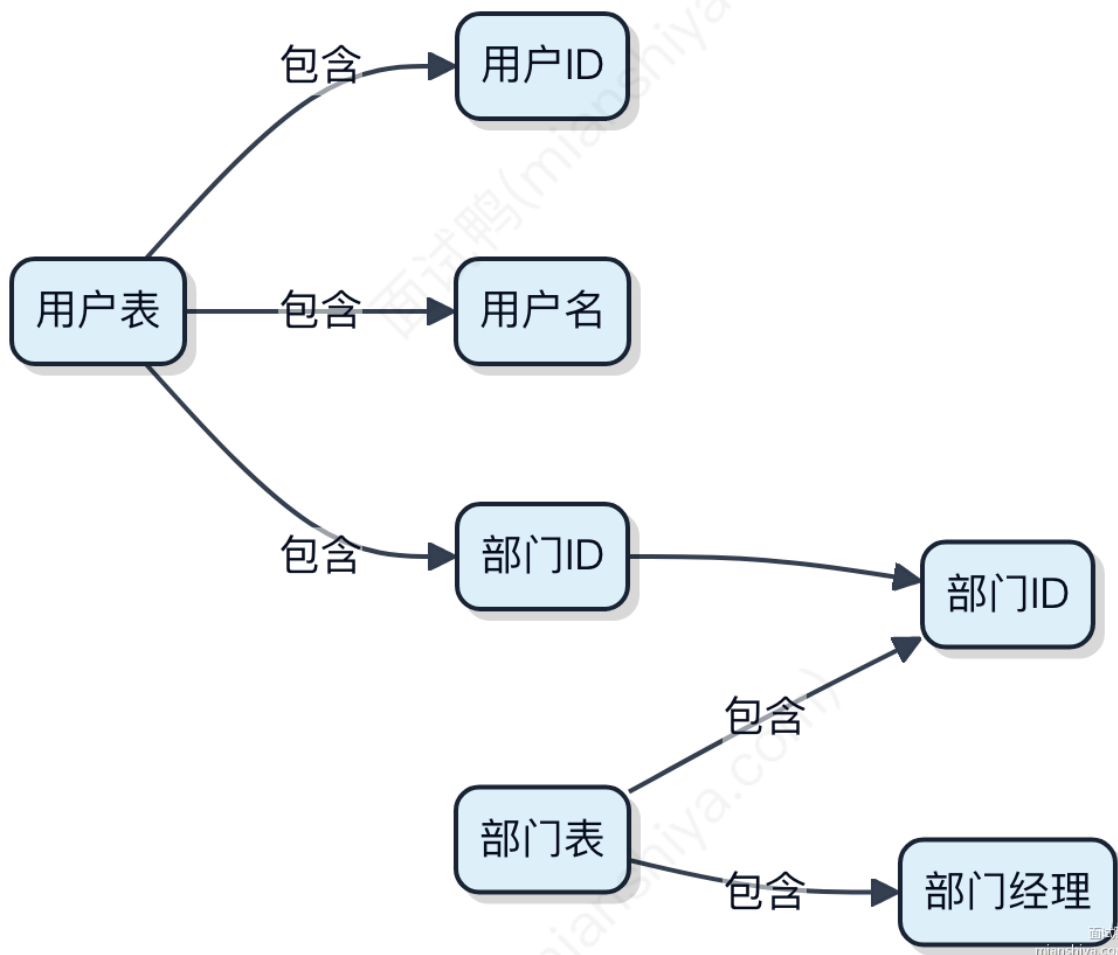
总的来说，函数是工具，用对地方提效，滥用就成慢查询根源了。

数据库的三大范式是什么？

第一范式说白了就是字段不能再分，每个列都得是原子的。比如你存个地址“北京市朝阳区”，后面要按城市查就不方便。要是拆成省、市、区三个字段，才算符合第一范式。

第二范式建立在第一范式基础上，要求所有非主键字段完全依赖于整个主键。这主要出现在联合主键场景里。比如说订单明细表，主键是订单ID加商品ID，那数量、单价这些字段就得同时依赖这两个主键。但像客户姓名这种只依赖订单ID的，就得拎出去单独建表，不然就存在部分依赖。

第三范式更进一步，非主键字段之间不能有依赖关系，都得直接依赖主键。举个例子，用户表里有部门ID和部门经理两个字段，部门经理其实是通过部门ID间接确定的，这就产生了传递依赖。正确的做法是把部门经理信息放到部门表里，用户表只保留部门ID作为外键。



总的来说，三大范式层层递进，目的是减少数据冗余、避免更新异常。不过实际项目中像TiDB或者一些OLAP系统，为了查询性能经常做反范式设计，比如预关联宽表。所以范式更像是理论指导，具体还得看业务场景。

什么是游标 Cursor 分页？相比传统 LIMIT OFFSET 分页有什么优势？

游标分页不是靠页码，而是通过上一页最后一个记录的某个唯一且有序的字段值（比如 ID 或时间戳）作为“锚点”，来获取下一页数据。

传统 LIMIT OFFSET 分页在深分页时性能很差。比如 `LIMIT 10000, 20`，数据库得先扫描前 1 万条数据，再扔掉它们，只返回第 10001 到 10020 条。数据量一大，这种查询会越来越慢，甚至拖垮数据库。

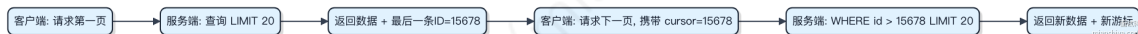
游标分页直接跳过前面的扫描。假设按 ID 升序排列，上一页最后一条的 ID 是 15678，那下一页就是：

```
SELECT * FROM orders
WHERE id > 15678
ORDER BY id
LIMIT 20;
```

这条查询能走索引，**压根不经过**前面的数据，响应基本稳定在毫秒级。

不过它也有局限。没法直接跳转到第 100 页，只能“下一页”这样顺滑滚动。适合无限加载场景，比如微博、朋友圈这类产品用的就是类似机制。Redis 的 SCAN 命令也是游标思想的应用。

另外，如果排序字段有重复值，可能需要组合字段做游标，避免漏数据。比如 `(created_at, id)` 联合定位。



MySQL 中的事务隔离级别有哪些？

事务隔离级别是为了解决并发场景下多个事务同时操作数据时产生的问题。MySQL 支持四种标准隔离级别，每种解决的问题和带来的副作用都不同。

读未提交 (Read Uncommitted)

一个事务能读到另一个事务还没提交的数据。这种情况最容易出问题，比如读到了“脏数据”，也就是对方事务回滚了，你读到的就是废的。一般没人用这个级别。

读已提交 (Read Committed)

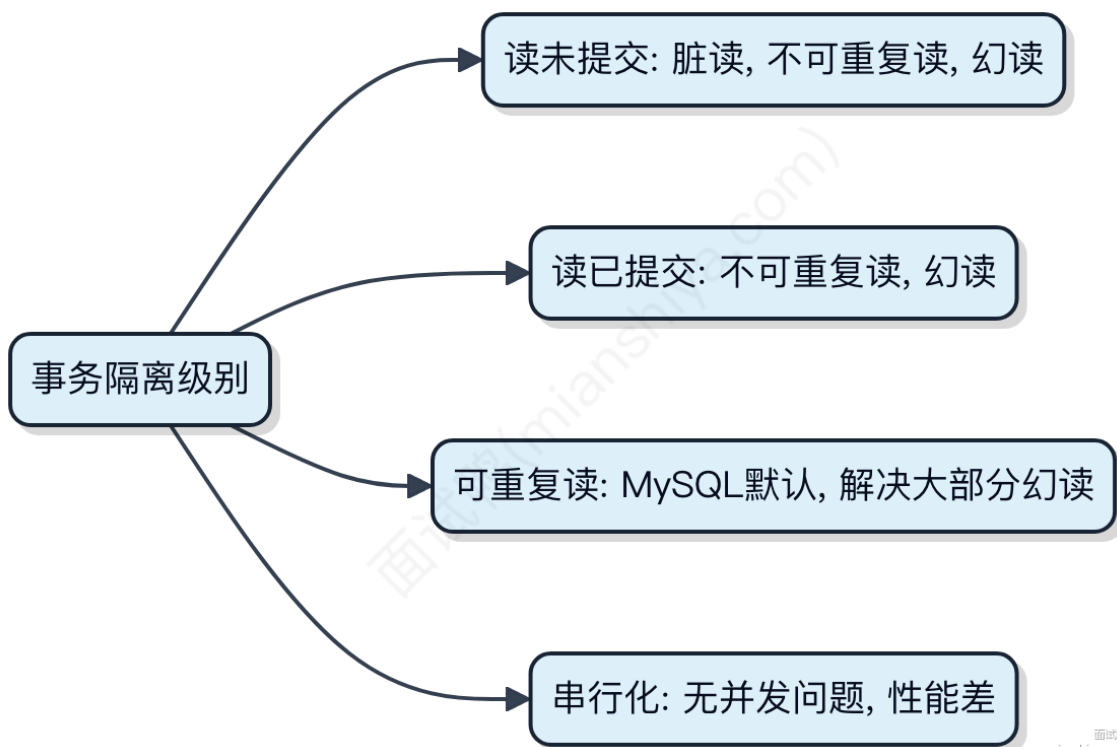
只能读到已经提交的数据，解决了脏读。但会出现不可重复读，就是同一个事务里两次读同一行数据，结果不一样，因为中间有别的事务提交了修改。PostgreSQL 默认就是这个级别。

可重复读 (Repeatable Read)

MySQL 的默认隔离级别。保证在同一个事务中多次读取同一数据结果一致，解决了不可重复读。通过 **MVCC**（多版本并发控制）实现快照读，普通查询不加锁。但理论上还可能存在幻读问题，不过 InnoDB 在这个级别通过间隙锁解决了大部分幻读场景。

串行化 (Serializable)

最高的隔离级别，所有事务排着队执行。读会加共享锁，写加排他锁，完全避免并发问题。但性能最差，吞吐降得厉害，只有对数据一致性要求极端严格的场景才会考虑。



实际业务中，基本都在用读已提交或可重复读。选择哪个看场景，比如金融系统可能倾向可重复读，而高并发读多写的系统可能选读已提交更合适。

MySQL一张表最多可以有多少列？

MySQL 单表的列数限制不是固定的，它受多个因素影响，最终取决于存储引擎和数据类型的组合。

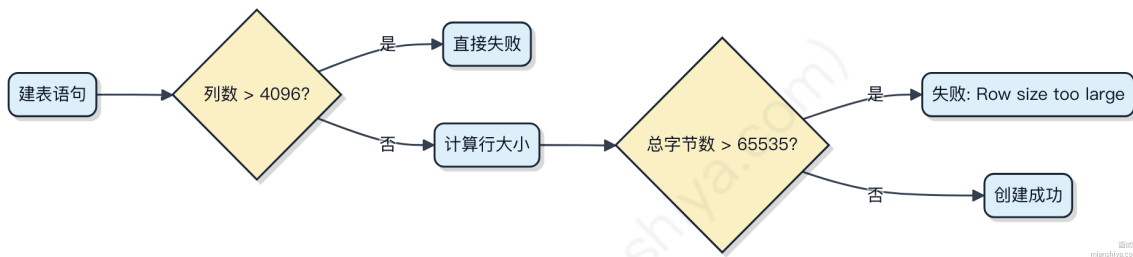
- 1) 对于 InnoDB 引擎来说，逻辑上支持最多 **1017 列**（MySQL 5.7 及之后版本），如果是 MySQL 8.0.34+，这个上限提升到了 **4096 列**。但这是理论最大值，实际能建多少列还卡在另一个硬限制上：行大小（row size）。
- 2) 所有列的长度加起来不能超过单行最大 **65,535 字节**。比如你定义很多 `VARCHAR(65535)`，其实根本存不下，因为一行压根不经过额外开销就超了。像 `CHAR`、`BLOB`、`TEXT` 这些类型也都会占用这部分空间，尤其是变长字段还有额外 1-2 字节记录长度。
- 3) 另外每个索引列也有影响，InnoDB 每张表最多支持 64 个二级索引，每个索引最多包含 16 个列，这也会间接约束设计。

举个例子：

```
CREATE TABLE t (  
  c1 int,  
  c2 varchar(100),  
  -- ... 一堆字段  
  c1000 varchar(10)  
);
```

看着没问题，但如果总字段太多且类型偏大，建表时就会报错 `Row size too large`。

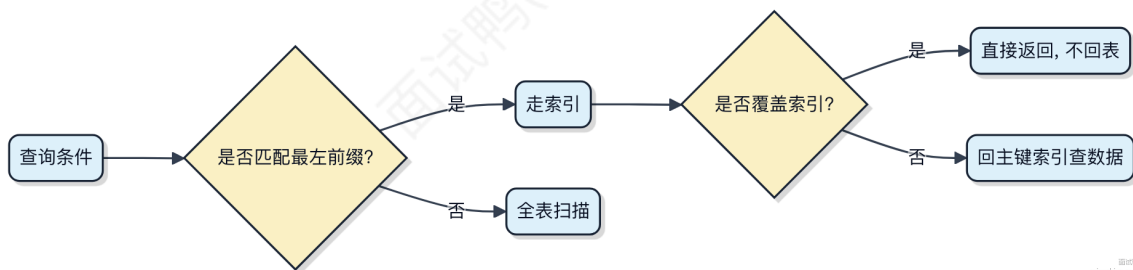
所以实践中，超过几百列就得警惕了。一般搞不定业务模型才需要这么多列，通常建议拆成宽表 + 属性表，或者用 JSON 字段收拢低频属性，别硬刚一长串 nullable 字段。



在 MySQL 中建索引时需要注意哪些事项?

索引不是建得越多越好，搞不好反而拖慢写入性能。每条 INSERT、UPDATE 都要维护索引结构，尤其是 B+ 树的分裂合并，成本不低。

- 1) 选择区分度高的列作为索引键。比如用户表用 `user_id` 比用 `gender` 强多了，后者只有两个值，查起来几乎全表扫。
- 2) 联合索引注意最左前缀原则。比如你建了 `(a, b, c)` 的组合索引，查询条件必须从 `a` 开始，只查 `b` 或 `c` 压根不经过这个索引。
- 3) 避免在索引列上做计算或函数操作。像 `WHERE YEAR(create_time) = 2023` 这种写法，即使 `create_time` 有索引也用不上，得改成范围查询。
- 4) 覆盖索引能显著提升性能。如果查询字段都在索引里，MySQL 直接从 B+ 树的叶子节点取数据，不用回表查主键索引，减少 IO。
- 5) 字符串字段别全建索引，太占空间。比如 `VARCHAR(255)` 可以只对前 10 个字符建前缀索引，但得保证前缀区分度够高。
- 6) 大表加索引要小心锁表。可以用 `ALTER TABLE ... ALGORITHM=INPLACE, LOCK=NONE`，或者借助 `pt-online-schema-change` 工具在线操作。
- 7) 定期检查冗余和未使用索引。`information_schema.statistics` 能看索引使用情况，长时间没被用到的可以考虑删掉。



什么是数据全量同步和增量同步? 它们各有什么优缺点?

数据同步一般分两种模式，全量同步和增量同步，选哪种得看场景。

全量同步就是把源库的所有数据一次性拉过来，不管目标端有没有。比如每天凌晨用DataX把MySQL的订单表全量导出到数仓一次。好处是逻辑简单，不容易出错，适合数据量小或首次初始化。但问题是数据量一大就搞不定，网络带宽和数据库压力都扛不住，像1000万条记录每次扫一遍，IO直接打满。

增量同步只传变化的数据，通常是基于binlog、timestamp字段或CDC工具。比如用Flink CDC监听MySQL的变更日志，只把新增或修改的行发到Kafka。这种方式资源消耗低，延迟也小，能实现准实时同步。但复杂度高，要处理断点续传、顺序一致性、中间环节丢数据等问题，维护成本明显更高。

实际项目里常见的是组合打法。第一次做**全量同步**，之后一直走**增量同步**。像阿里云DTS就是这样设计的。如果中间断了太久，增量追不上，系统会自动触发重新全量。

要不要加图？这里流程清晰，文字足够说明。

1) 全量：源库 → 全表扫描 → 目标库（覆盖） 2) 增量：源库 → 捕获变更（binlog） → 消息队列 → 目标库（追加/更新）

两者本质是吞吐与延迟的权衡。小数据+可靠性优先选全量，大数据+实时性要求选增量。

MySQL 索引的最左前缀匹配原则是什么？

当你的 SQL 查询条件用到了联合索引，但不是从第一个字段开始，也不是连续使用时，能不能命中索引，就得看最左前缀匹配原则。

这条规则的意思是，MySQL 会一直向右匹配直到遇到范围查询（比如 >、<、BETWEEN、LIKE）或者没有使用的列为止，中间不能断。

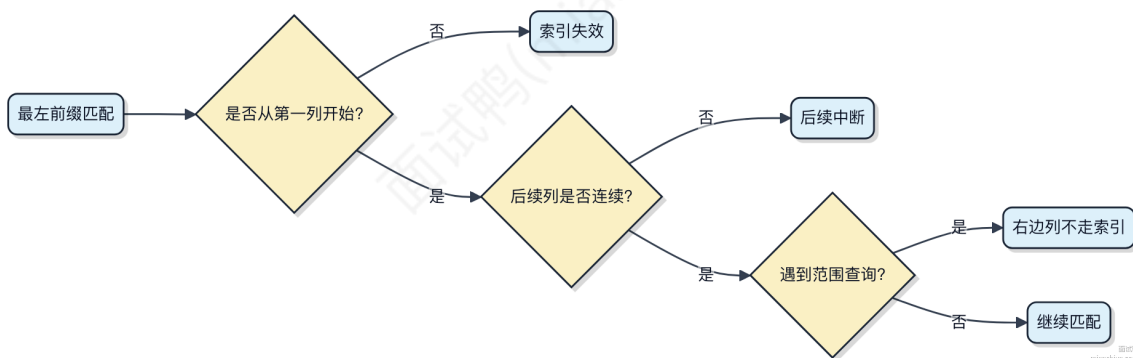
举个例子，你有个联合索引 (a, b, c)，那么下面这些查询能走索引：1) 只查 a 2) 查 a 和 b 3) 查 a、b、c

但如果查的是 b 单独，或者 b 和 c，压根不经过 a，那这个联合索引就用不上。因为不符合“最左”这个前提。

特别注意范围查询的影响。比如 `WHERE a = 1 AND b > 2 AND c = 3`，虽然三个字段都用了，但因为 b 是范围，c 就没法再利用索引了。也就是说，**范围查询后面的列不会走索引**。

实际开发中，像 user_id 和 create_time 这种高频组合，如果你把 create_time 放前面，而查询经常是等值查 user_id + 范围查时间，那索引顺序错了就搞不定查询性能。

所以建联合索引的时候，顺序很关键：等值查询放前面，范围查询放后面，并且尽量让高频字段靠左。



什么是 CDC (Change Data Capture)？常见的 CDC 工具有哪些？

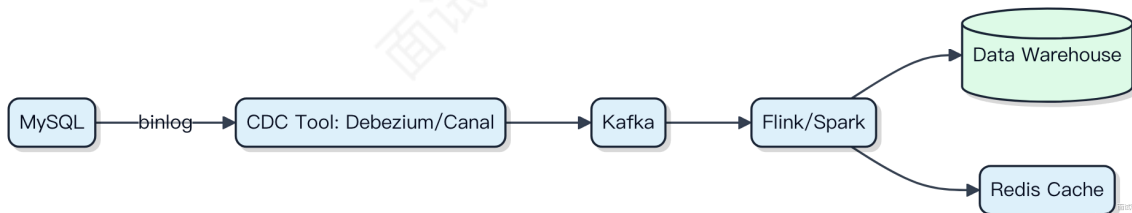
CDC 的本质是捕获数据变更事件，而不是定期轮询快照。数据库的事务日志比如 MySQL 的 binlog、PostgreSQL 的 WAL 天然就是变更流，**CDC 工具**就是把这些日志实时解析成可消费的消息。

这类工具不依赖业务代码侵入，能以极低延迟把增删改操作同步到下游，典型场景是保持缓存与数据库一致、数据仓库实时入仓、微服务间解耦。如果靠定时任务扫表，延迟高还容易把数据库搞崩。

常见开源方案有几种路线：

- 1) 基于数据库日志解析，Debezium 是主流选择，它用 Kafka Connect 框架做分布式采集，支持 MySQL、PostgreSQL、Oracle 等多种源，能把每一行变化发到 Kafka Topic 里。
- 2) 阿里开源的 Canal 只针对 MySQL，模拟 slave 拉取 binlog，适合纯 Java 技术栈的公司集成，像阿里内部早期就用它做订单数据同步。
- 3) 商业产品如 Oracle GoldenGate 功能更强，但成本高；现代数仓场景也有 Flink CDC 直接嵌入计算引擎，用 SQL 就能写变更管道。

代码层面，Debezium 的配置核心是定义 connector 的 source 属性，指定数据库地址、binlog 位点、加密方式等，启动后自动生成数据流。



注意点：DDL 变更处理要小心，字段删除可能导致消息格式不兼容；还有断点续传必须依赖可靠的 offset 存储。

什么是批量数据入库？相比单条插入有什么优势？

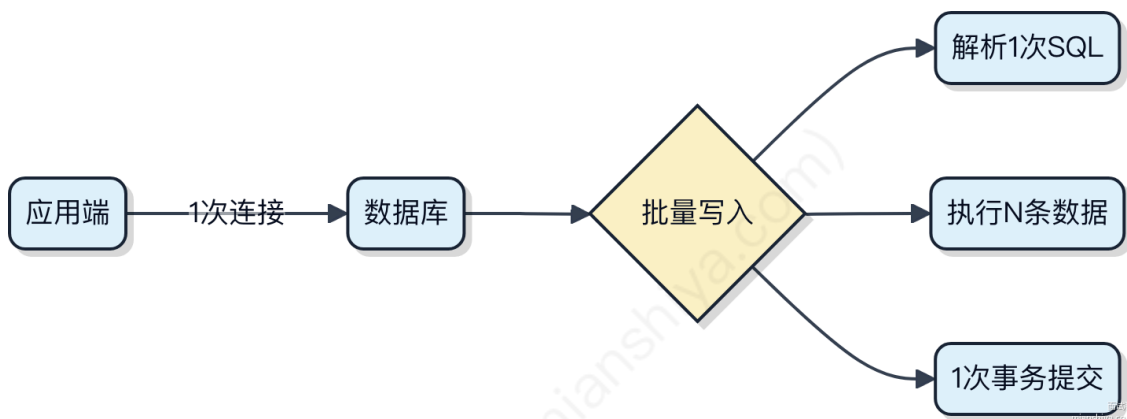
批量数据入库指的是把一批数据一次性写入数据库，而不是一条一条地提交。这个操作在数据导入、日志处理、ETL 场景里特别常见，比如用 **MyBatis Plus** 的 `saveBatch` 方法，或者 JDBC 直接执行 `addBatch()` + `executeBatch()`。

单条插入每次都要走一次网络往返、事务开销、日志刷盘，假设你要插 1 万条数据，就得连着执行 1 万次 `prepare`、`execute`、`commit`。而批量入库能把这 1 万条打包成 100 个批次，每个批次 100 条，实际执行次数直接降到 100 次。

最核心的优势是 **性能提升**。一般情况下，批量插入的吞吐量能比单条高 5~10 倍，甚至更多。原因也很直接：网络交互少了，数据库的 SQL 解析压力小了，事务提交次数也大幅降低。

但也不是越大越好。如果单批次达到几千上万条，可能引发大事务问题，内存占用飙升，甚至导致连接超时或锁表。一般来说，每批控制在 200~500 条比较稳妥，具体要看数据库配置和硬件能力。

另外，像 **MySQL** 在使用 `rewriteBatchedStatements=true` 参数后，会把批量插入重写成 `INSERT INTO ... VALUES (...), (...), (...)` 这种多值形式，效率更高，这种优化单条压根不经过。



MySQL 的索引类型有哪些？

MySQL 的索引类型主要看存储引擎支持，最常用的是 InnoDB，它的索引结构基于 B+ 树。

1) 主键索引（聚簇索引）

行数据和主键一起存储在 B+ 树的叶子节点，物理顺序和主键顺序一致。每张表只能有一个，查询效率最高，范围查询也快。

2) 二级索引（普通索引）

非主键列上的索引，B+ 树叶子节点存的是主键值。查非主键字段会先走二级索引找到主键，再回表查聚簇索引，这个过程叫回表。

3) 唯一索引

和二级索引类似，但要求列值唯一，允许一个 NULL。创建唯一约束时自动建这种索引。

4) 联合索引

多个列组合的索引，遵循最左前缀原则。比如 (a, b, c) 能命中 a、(a,b)、(a,b,c)，但不能用 b 单独查。

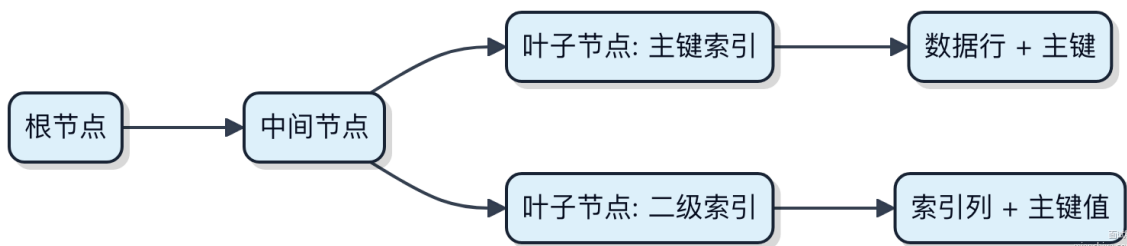
5) 全文索引

用于文本内容的关键词搜索，支持 MATCH AGAINST 语法。MyISAM 和 InnoDB 都支持，但使用场景不多，一般用 Elasticsearch 或 OpenSearch 替代。

6) 前缀索引

对字符串前 N 个字符建索引，节省空间。比如 index(email(10))，但会影响区分度，可能增加扫描行数。

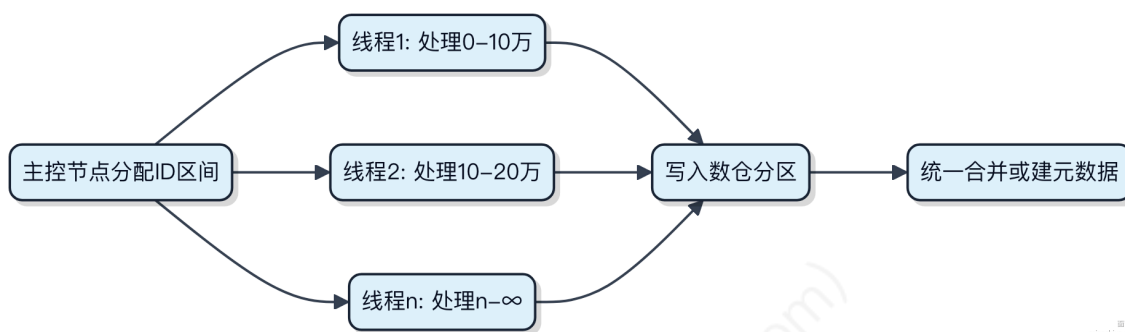
覆盖索引是个重要优化点。如果查询的字段都在索引里，压根不经过回表，直接从二级索引返回数据，速度很快。比如给 (name, age) 建联合索引，select name, age from t where name=? 就能覆盖。



多线程并发同步数据时（数据库的数据同步到数仓中）需要注意什么问题？

多线程做数据库到数仓的数据同步，本质是在解决**一致性和性能**的平衡问题。你得先想清楚数据从哪来、怎么拉、怎么写。

- 1) 拉取数据时别直接 `SELECT * FROM table`，大表一查就卡住，还可能把数据库连接池搞崩。一般用自增 ID 或时间戳做分片，比如按 ID 范围切任务，每个线程处理一段，这样能并行拉数据。如果是 MySQL，可以考虑 `WHERE id BETWEEN ? AND ?` 配合索引。
- 2) 多个线程往数仓写数据，必须避免重复写或漏写。常见做法是每个线程处理的数据块带唯一标识，在数仓侧做幂等，比如 Hive 或 ClickHouse 写入前先检查分区是否存在，或者用事务型表引擎（如 Iceberg）提交时做原子替换。
- 3) 共享状态要小心。比如有个全局计数器记录同步进度，多个线程更新它就得加锁，但其实更好的方式是让每个线程独立汇报进度，由协调者汇总，压根不经过临界区。
- 4) 异常处理不能少。某个线程挂了，任务得能重试，但又要防止重复同步。通常把任务状态存到外部存储，比如 ZooKeeper 或数据库里的任务表，失败后可恢复。
- 5) 别忽视数据库的负载。并发太大，源库的 CPU、IOPS 可能扛不住。一般控制并发线程在 10~20 个以内，配合限流，像 Canal 或 DataX 实际也是这么干的。



MySQL 中 TEXT 类型最大可以存储多长的文本？

MySQL 的 TEXT 类型不是单一类型，而是一组用于存储大文本的字段类型，不同变种容量差别很大。

- 1) TINYTEXT 最大 255 字节，和 VARCHAR(255) 上限一样，但存储方式不同。
- 2) TEXT 最大 65,535 字节（约 64KB），实际能存的字符数取决于字符集。比如用 utf8mb4，一个字符占 4 字节，最多存 16383 个字符左右。
- 3) MEDIUMTEXT 支持到 16,777,215 字节（约 16MB），适合存文章内容、日志片段这类中等长度文本。
- 4) LONGTEXT 能到 4,294,967,295 字节（约 4GB），基本能满足绝大多数大文本场景。

这些类型的数据**不存储在行内**，而是把实际内容放在额外的溢出页，行里只存指向它的指针。这样一来，虽然能存得下大文本，但访问时会有一次间接寻址，性能比 VARCHAR 差一些。

如果字段经常用来做 WHERE 查询或 ORDER BY，尽量别用 TEXT，因为索引受限。比如普通索引最多前缀 767 字节（或 3072 字节，取决于 innodb_large_prefix 配置），全文检索可以用 FULLTEXT 索引，像 MySQL 自带的全文索引或者接入 Elasticsearch 更合适。

代码上建表时写法很简单：

```
CREATE TABLE articles (  
  id INT PRIMARY KEY,  
  content LONGTEXT  
);
```

一般选哪种，得看业务数据量级。存个用户简介用 TEXT 绰绰有余，存小说章节建议 MEDIUMTEXT，搞文档归档系统可能就得 LONGTEXT。

MySQL 中 AUTO INCREMENT 列达到最大值时会发生什么？

当一张表的 AUTO_INCREMENT 列达到其数据类型定义的最大值时，数据库没法再生成新的唯一自增值，下一次插入就会直接报错，语句中断。

比如你用的是 INT 类型，最大值是 2147483647。一旦自增到这个数，再执行 INSERT，MySQL 就会抛出类似“Duplicate entry '2147483647' for key 'PRIMARY'”的错误，其实是**自增溢出**，但表现像是主键冲突。

根本原因是，自增机制在尝试加 1 时发生了整数溢出，值卡住不变了，导致后续所有插入都试图用同一个 ID，自然就撞了主键。

1) 不同整数类型的上限得心里有数：

- TINYINT 最大 127（有符号）或 255（无符号）
- SMALLINT 是 32767 / 65535
- MEDIUMINT 是 8388607 / 16777215
- INT 是 2147483647 / 4294967295
- BIGINT 是 9.2e18 / 1.8e19

小表用 INT 一般够用，但高并发写入的场景，比如订单、日志类系统，几年下来可能就逼近极限，这时候就得用 BIGINT。

2) 预防这事，其实很简单：设计阶段就评估数据增长量。日均百万写入的话，一年就是 3.6 亿，不到 12 年就能把有符号 INT 耗尽。所以新项目直接用 BIGINT UNSIGNED 更省心。

3) 已经快满了怎么办？可以改列类型，但 ALTER TABLE 在大表上是**脏活累活**，会锁表。可以用在线工具如 pt-online-schema-change，或者提前分库分表，把压力分散掉。

MySQL 二级索引有 MVCC 快照吗？

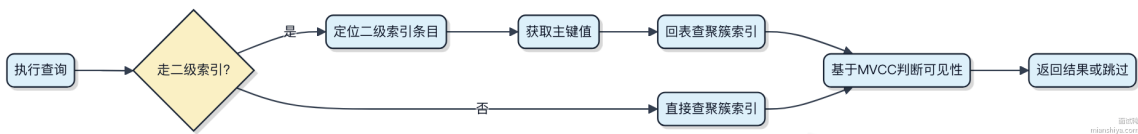
MySQL 的 InnoDB 引擎里，MVCC 主要靠聚簇索引中的隐藏字段和 undo log 来实现。你查主键的时候，InnoDB 能通过这些机制判断某条记录对当前事务是否可见。

但二级索引本身不带 MVCC 信息。它叶子节点存的是主键值，不是完整行数据。当你走二级索引查找时，会先定位到对应的主键值，再回表去聚簇索引里查那一行。**真正的可见性判断**，是在回表后的聚簇索引查询阶段才做的。

也就是说，二级索引只是一个“跳板”。它不保存 trx_id、roll_ptr 这些 MVCC 所需的元数据，压根不参与快照读的判定逻辑。查询过程中，虽然二级索引页也会被加载进 buffer pool，但它不会像聚簇索引那样应用 undo log 构建历史版本。

所以你在读二级索引时，看到的是索引条目的最新状态，但这不代表你能直接看到对应数据行的最新版本——最终能不能返回这条记录，还得看回表后在聚簇索引中检查 MVCC 条件的结果。

举个例子，你在一个已删除但未提交的记录上查二级索引，依然能走到那个索引项，但回表时发现主键记录已被标记删除且当前事务看不到它，结果就是不返回。



什么是数据库的视图？

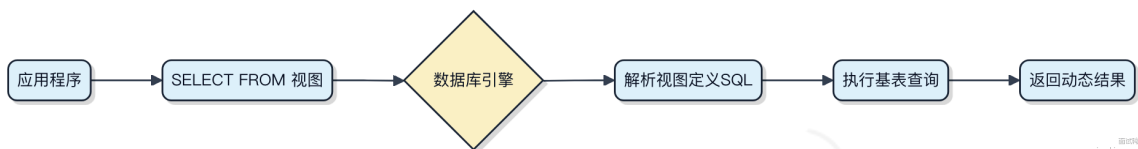
数据库里的视图，说白了就是一条被命名的 SQL 查询语句，它不存实际数据，只存逻辑。你查视图的时候，数据库才临时执行背后的 SQL 把结果拼出来。

这种设计主要解决几个问题：一是简化复杂查询，比如多表关联、聚合计算这些脏活累活封装到视图里，业务方直接 `SELECT * FROM v_user_order` 就行；二是做权限隔离，比如用户表有敏感字段 `id_card`，你可以建个视图只暴露 `user_name` 和 `age`，让报表系统只能查这个视图，压根不经过原始表。

还有一点是保持接口稳定。底层表结构要是改了，比如把 `user_info` 拆成 `user_base` 和 `user_ext`，只要视图的字段输出不变，上层应用就不用改代码，能平滑过渡。

不过得注意，视图是查的时候才计算的，所以叫“虚表”。频繁查大范围数据的视图可能拖慢查询，特别是嵌套好几层的视图，执行计划会变得很复杂，搞不定的时候就得拆开优化。

有些数据库像 MySQL 支持对视图做 `CREATE INDEX`，但其实是给基表加索引，本身不能独立建索引。PostgreSQL 有物化视图，可以提前把结果存下来，查得快，但数据不是实时的。



什么是数据库的游标？

数据库里的游标，你可以把它看作一个“查询结果的指针”。执行一条 `SELECT` 语句后，数据库返回一堆数据，但程序没法一口气全吃进去。这时候就需要游标，一点一点地从结果集里取数据。

- 1) 它本质上是一个上下文句柄，保存了当前查询的状态，比如“现在读到第几行了”、有没有下一行。
- 2) 典型的使用场景是存储过程或批量处理，比如在 MySQL 存储过程中用 `DECLARE CURSOR` 声明，然后 `FETCH` 逐行读取，适合处理几万甚至上百万条记录的后台任务。
- 3) 代码上像这样：

```

DECLARE emp_cursor CURSOR FOR SELECT id, name FROM employees;
OPEN emp_cursor;
FETCH NEXT FROM emp_cursor INTO @id, @name;
-- 循环处理
CLOSE emp_cursor;
  
```

其实大多数 Web 应用压根不经过游标，JDBC 或 ORM 比如 MyBatis 默认是一次性拉取结果集，底层已经把数据读进内存了。只有在数据量太大、内存扛不住的时候，才会考虑用服务器端游标，让数据库分批吐数据。

容易搞混的一点是：客户端分页不是游标。真正的游标是在数据库服务器上保持连接状态，会占用资源，用完必须关掉，否则可能拖慢数据库。

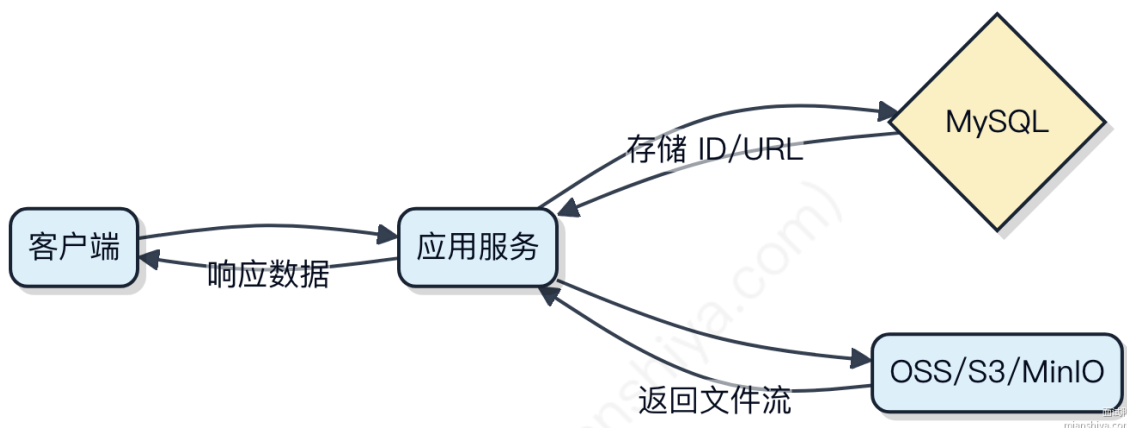


为什么不推荐在 MySQL 中直接存储图片、音频、视频等大容量内容？

MySQL 这种关系型数据库，设计初衷是处理结构化数据，像整数、字符串、时间这些固定或小长度的类型。你要是硬塞图片、音频、视频进去，基本都得用 BLOB 字段，但这会带来一堆麻烦。

- 1) 存储成本翻倍。同样的文件，在 MySQL 里存一遍，备份还得再存一遍，占空间不说，备份时间也成倍涨。一个 10MB 的文件，每天增量 1000 个，一天就是 10GB，磁盘搞不定。
- 2) 性能直接拖垮。大文件读写要占大量内存和网络带宽，MySQL 的 buffer pool 会被污染，缓存命中率暴跌。本来跑得好好的查询，全被这些大对象拖慢了。
- 3) 扩展性几乎为零。数据库主从同步时，大 BLOB 会卡住复制线程，导致延迟飙升。分库分表更别提，文件根本没法拆。

真要存这类内容，主流做法是把文件扔到 **对象存储** 里，比如阿里云 OSS、AWS S3 或 MinIO。数据库里只留个 URL 或文件 ID。访问的时候应用先查 DB 拿地址，再去对象存储拿文件。这套组合拳在互联网公司已经是标配。



这么拆，数据库专注元数据管理，对象存储扛住海量文件，各干各的，系统才能撑得住。

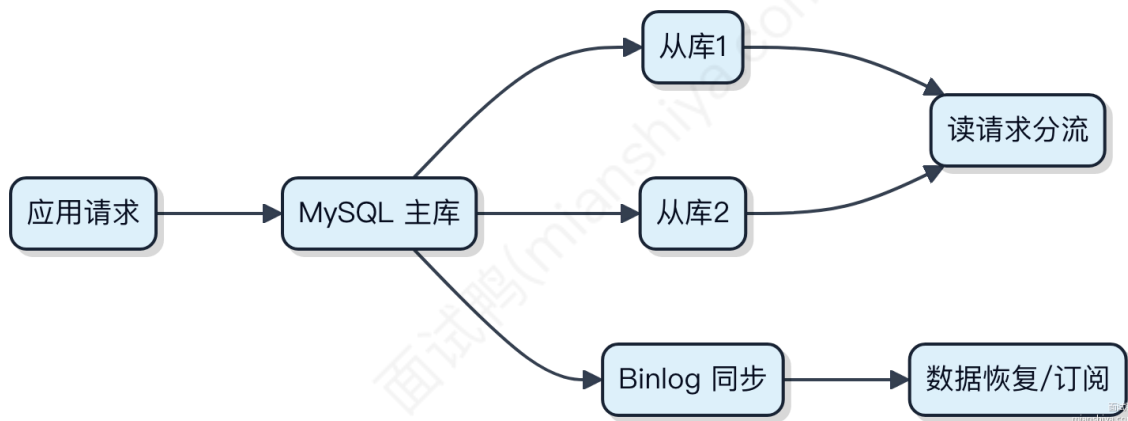
相比于 Oracle，MySQL 的优势有哪些？

MySQL 和 Oracle 都是关系型数据库，但定位和设计哲学差得挺远。MySQL 起初就是奔着轻量、快、易部署去的，尤其适合互联网场景。

- 1) 开源免费是硬道理。MySQL 社区版完全免费，像阿里、腾讯早期都是靠它省下几百万授权费。Oracle 一套企业版动辄几十万，还得按核收费，小公司压根扛不住。
- 2) 性能侧重点不同。MySQL 在读多写少、高并发的简单查询场景下响应更快，比如商品列表页、用户评论这些接口，QPS 轻松上万。它的 InnoDB 引擎支持行级锁和 MVCC，已经能覆盖 90% 以上的业务需求。
- 3) 架构轻，运维成本低。MySQL 单实例资源占用小，512MB 内存都能跑起来。配合主从复制 + MHA 或 Sentinel，故障切换也够用。相比之下 Oracle 是“大块头”，依赖专用工具和 DBA 精细调优，人力成本高一截。

4) 生态适配好。和 LAMP/LEMP 栈天然契合，Docker 部署一条命令就起一个实例。云厂商 RDS 支持也到位，阿里云、AWS 都优先优化 MySQL 兼容性。

不过复杂事务、强一致性报表、超大数据量 OLAP 场景，Oracle 的 RAC、Data Guard、分区能力还是更稳。但大多数 Web 应用根本用不到这些，反而被拖累。



总的来说，MySQL 赢在**快速迭代、低成本、够用就好**。互联网要的是快速上线、扛住流量，而不是一开始就搞一套重型系统把自己绑死。

在什么情况下，不推荐为数据库建立索引？

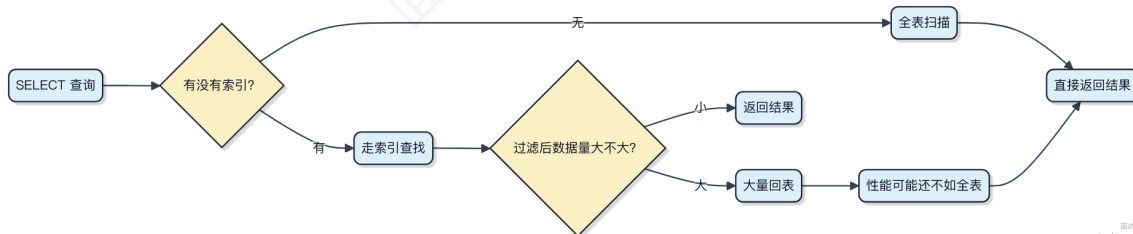
数据量小的时候，比如表里就几百条记录，查询全表扫描可能比走索引还快。因为索引本身要维护 B+ 树结构，读写都有额外开销，数据少的情况下这层跳转反而拖慢速度。

频繁更新的字段也不适合建索引。像用户余额、订单状态这种改动特别勤的列，每次变更都要同步更新索引树，**B+ 树**为了维持平衡会触发分裂或合并，写放大问题严重，容易成为性能瓶颈。

区分度低的字段建了也白建。比如性别只有男/女，或者状态码就几个固定值，查索引后还是要回表扫一大片，优化器干脆就走全表了。MySQL 一般预估扫描行数超过 20% 就不会用索引。

大文本字段别直接建普通索引。VARCHAR(1000) 这种往索引里一放，一页存不了几条，树层级拉得老高，IO 效率暴跌。真要搜可以用前缀索引，但得权衡截取长度和冲突率。

- 1) 联合索引注意最左匹配原则，中间断了就用不上
- 2) NULL 值多的字段效果差，索引对空值处理比较麻烦
- 3) OR 条件如果非主键字段没覆盖到，索引也可能失效



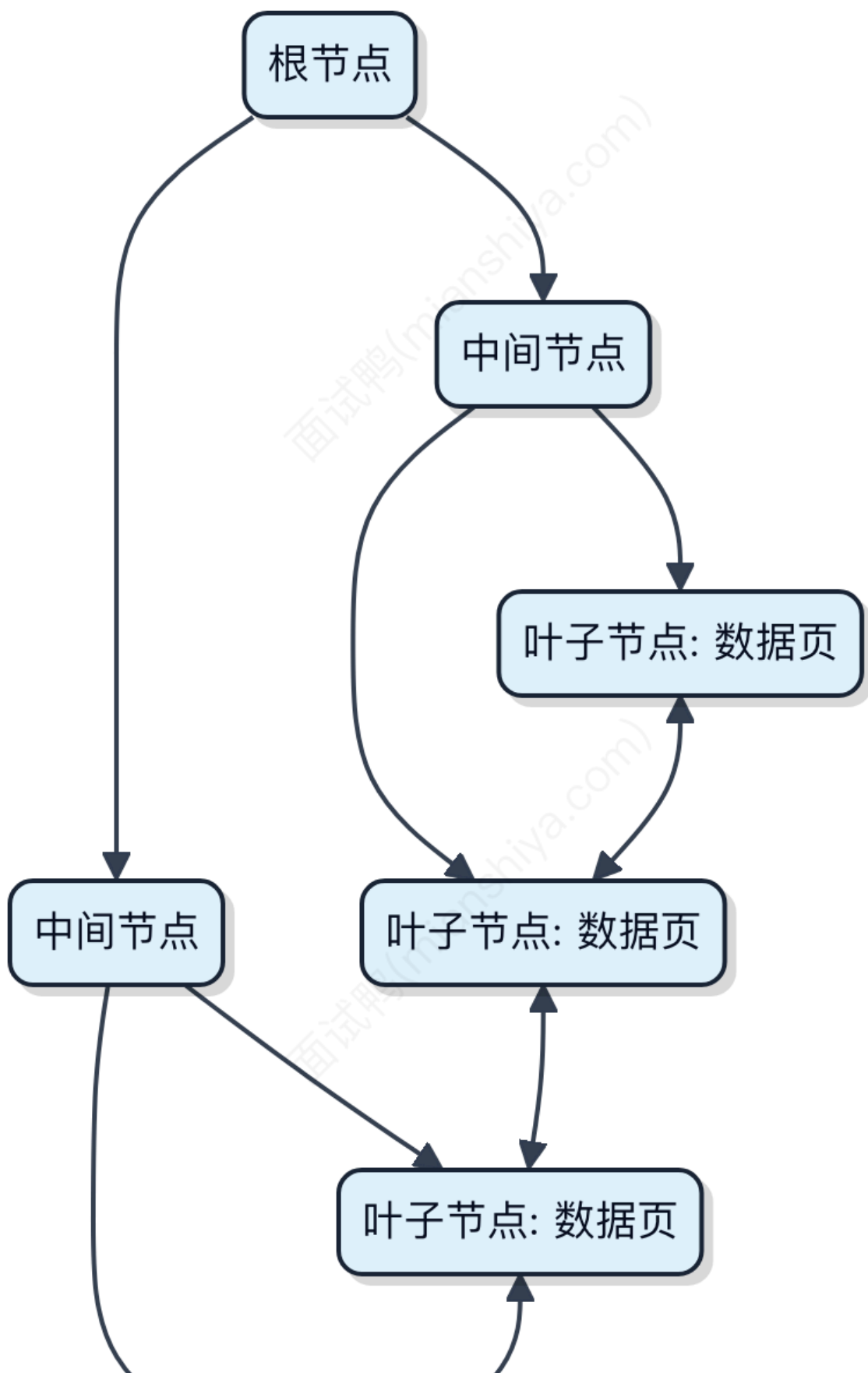
为什么 MySQL 索引用的是 B+ 树而不是红黑树？

MySQL 的索引选型，本质是为了解决磁盘 I/O 和数据量大的问题。内存结构像红黑树虽然查询快，但压根不适用于数据库这种基于磁盘存储的系统。

B+ 树的优势在于它的**多路平衡**和**层级少**。比如一个 3 层的 B+ 树，能存上千万甚至上亿的数据，而每次查询最多只需要 3 次磁盘 IO。相比之下，红黑树在相同数据量下层数会高得多，可能要 20~30 层，意味着查一次要访问磁盘二三十次，这谁扛得住。

而且 B+ 树的所有数据都存在叶子节点，非叶子节点只存索引，这样一页能放更多关键字，进一步减少树的高度。叶子节点之间还有双向指针，范围查询特别高效，像 `WHERE id BETWEEN 100 AND 200` 这种场景，直接链表遍历就行。

反观红黑树，每个节点只能存两个子节点，树会比较“瘦高”，节点分散，磁盘预读效果差，顺序访问也不友好。InnoDB 存储引擎就是靠 B+ 树来支撑海量数据下的高性能查找。



叶子节点: 数据页

面试鸭
mianshiya.com

简单说, B+ 树天生为磁盘设计, 一次读取能加载大量有序关键字, 利用局部性原理, 把真正的数据检索成本降到最低。

什么是 Write-Ahead Logging (WAL) 技术? 它的优点是什么? MySQL 中是否用到了 WAL?

WAL 的核心思路是: 在修改数据前, 先确保修改操作的**日志落盘**。只要日志持久化了, 数据页后续再刷盘也没问题。即使中途崩溃, 重启后靠日志重放就能恢复未写入的数据。

这种机制让数据库的随机写变成了日志的顺序写。磁盘或 SSD 对顺序写优化得多好, 你懂的。性能提升的关键就在这。

优点直接拉满: 1) 提升性能。写日志是追加操作, 比随机更新数据页快一个数量级。2) 保证持久性。只要日志 fsync 到磁盘, 事务就可以返回成功, 数据不会丢。3) 简化恢复。崩溃后用日志 replay 状态, 比检查整个数据文件高效多了。

MySQL 的 InnoDB 引擎就是靠这个撑起来的。它的 **redo log** 就是典型的 WAL 实现。你执行一条 UPDATE, InnoDB 先写 redo log buffer, 事务提交时刷到磁盘的 redo log 文件, 之后才异步刷脏页到 ibd 文件。

```
// 伪代码示意 WAL 流程
write_log_to_buffer(log);    // 写日志到内存
fsync_log_file();           // 日志落盘 (关键步骤)
apply_change_to_data_page(); // 数据页可以晚点刷
```

整个过程像极了记账: 不急着改总账本, 先在流水本上记一笔, 回头慢慢对账。这流水本就是 redo log。



你们生产环境的 MySQL 中使用了什么事务隔离级别? 为什么?

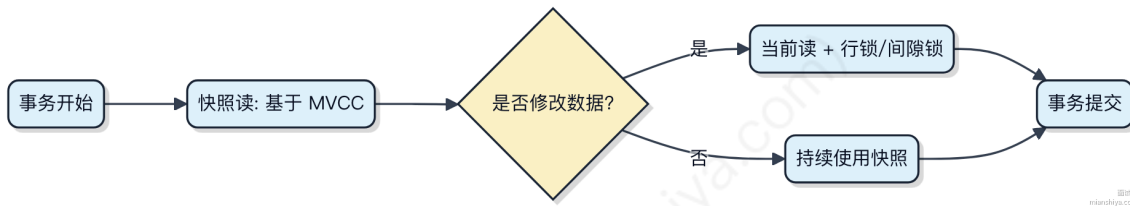
MySQL 默认的事务隔离级别是**可重复读** (Repeatable Read), 生产环境大多也直接用这个级别, 基本不改。

这个级别的特点是, 一个事务里头两次读同一行数据, 结果一定一样, 不会出现幻读或者不可重复读。InnoDB 通过 **MVCC** (多版本并发控制) 和间隙锁来实现这一点。比如你查了 `SELECT * FROM orders WHERE user_id = 100`, 后面再查一次, 哪怕别的事务插入了新订单, 你也看不到——除非他们提交了并且你的事务重新开始。

为什么不用更高的串行化? 太重了, 所有操作都加锁排队, 性能直接腰斩。而读已提交虽然并发高, 但会出现不可重复读, 像订单系统里头, 你第一次看到余额 100, 第二次变成 50, 用户肯定懵。

举个实际场景: 电商平台下单扣库存。事务开始读库存是 10, 中间执行逻辑, 最后扣减时还是按 10 算, 不能因为别人下单导致我这边莫名其妙变 9。可重复读就保证了这个一致性, 还不用牺牲太多并发能力。

代码上不需要额外设置，Spring 的 `@Transactional` 默认就会启用可重复读下的行为，前提是存储引擎是 InnoDB。



当然也有坑，比如用 `SELECT ... FOR UPDATE` 或者 `UPDATE` 语句时，会走当前读，这时候就可能看到别的事务提交的新数据，得注意逻辑顺序。总的来说，可重复读在一致性和性能之间拿捏得刚好，大多数业务扛得住。

SQL 中 select、from、join、where、group by、having、order by、limit 的执行顺序是什么？

SQL 语句的执行顺序不是按照书写顺序来的，而是有固定的逻辑处理步骤。理解这个顺序对写查询和调优都特别关键。

- 1) from: 先确定数据来源，把表加载进来，可能还会做表连接前的初步处理。
- 2) join: 接着处理 join，把关联表按 on 条件拼起来，生成一张临时的逻辑结果集。
- 3) where: 然后走 where 条件过滤，这一步能提前干掉大量不需要的行，提升效率。注意这里不能用聚合函数。
- 4) group by: 接下来按字段分组，每个分组对应一行结果，这时候才真正开始“汇总”数据。
- 5) having: 分组完后用 having 过滤分组结果，可以带 count、sum 这类聚合函数。
- 6) select: 到了这一步才选出要返回的字段，包括计算列。别名也是在这定义的，所以前面不能用别名。
- 7) order by: 然后排序，可以基于 select 里的别名来排，支持 asc/desc。
- 8) limit: 最后才限制返回条数，比如 limit 10 就拿前 10 条。



有个典型的坑是，在 where 里用了聚合函数，直接报错。因为聚合是在 group 阶段才发生的。还有就是 order by 能用别名，而 where 不行，原因就在执行顺序里。

MySQL 中 InnoDB 存储引擎与 MyISAM 存储引擎的区别是什么？

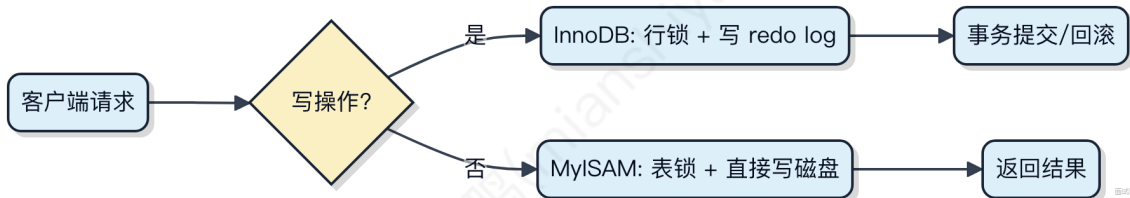
InnoDB 和 MyISAM 最大的区别在于事务支持和锁机制，这直接决定了它们的适用场景。

- 1) 事务处理上，InnoDB 支持完整的 ACID 事务，能回滚操作，数据一致性有保障。MyISAM 压根不支持事务，一旦写入出错只能手动修复。
- 2) 锁粒度方面，InnoDB 支持行级锁，高并发更新时冲突少，能扛住密集写操作。MyISAM 只有表级锁，只要写表就锁住整个表，写多的场景容易堵死。
- 3) 外键约束是 InnoDB 独有的功能，MyISAM 不识别外键定义，数据完整性得靠应用层来保证。

4) 查询性能要看场景。MyISAM 的 COUNT(*) 在没有 WHERE 条件时特别快，因为它把总行数存起来了。但加了条件还是得老老实实扫描。InnoDB 得实时算，但整体更稳。

5) 崩溃恢复能力差异大。InnoDB 有 redo log 和 undo log，机器宕机后能自动恢复。MyISAM 挂了之后表容易坏，得用 myisamchk 工具修。

现在新项目基本都用 InnoDB，像订单、账户这类要保证一致性的必须上 InnoDB。MyISAM 基本只在纯日志类、只读报表这种极少更新的场景还能见到。



MySQL 插入一条 SQL 语句，redo log 记录的是什么？

redo log 记录的是物理日志，不是 SQL 语句本身。它记的是数据页的修改细节，比如“在某个表空间的第几号数据页上，偏移量为多少的位置，改了几个字节，从什么值变成什么新值”。

这种设计是为了保证崩溃恢复时能重新应用这些改动，确保事务的持久性。即使数据库中途宕机，只要 redo log 持久化到磁盘了，重启后就能把未刷入磁盘的数据页变更重新做一遍。

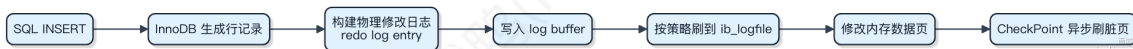
举个例子，执行一条 `INSERT INTO users(id, name) VALUES (1, 'Tom')`，InnoDB 不会把这条 SQL 记进 redo log，而是生成类似这样的记录：

“往表空间 ID=10 的第 5 号数据页，偏移 100 处插入一行，共 38 字节，内容是 0x01...”。

这个过程由存储引擎自己控制，**物理日志**、顺序写、循环写入日志文件组，性能远高于随机写数据文件。

redo log 是 InnoDB 实现 WAL (Write-Ahead Logging) 的关键。所有修改必须先写日志再改内存中的数据页，后续通过 checkpoint 机制异步刷盘。

- 1) 每个事务提交时会把生成的 redo 日志写入 log buffer
- 2) 根据 `innodb_flush_log_at_trx_commit` 配置决定是否立刻刷盘
- 3) 最终由后台线程批量将日志刷到磁盘上的 `ib_logfile` 文件中



MySQL 在设计表（建表）时需要注意什么？

建表不是随便定义几个字段就完事，得考虑数据怎么存、怎么查、怎么扩展。设计不合理，后期加索引都救不了性能。

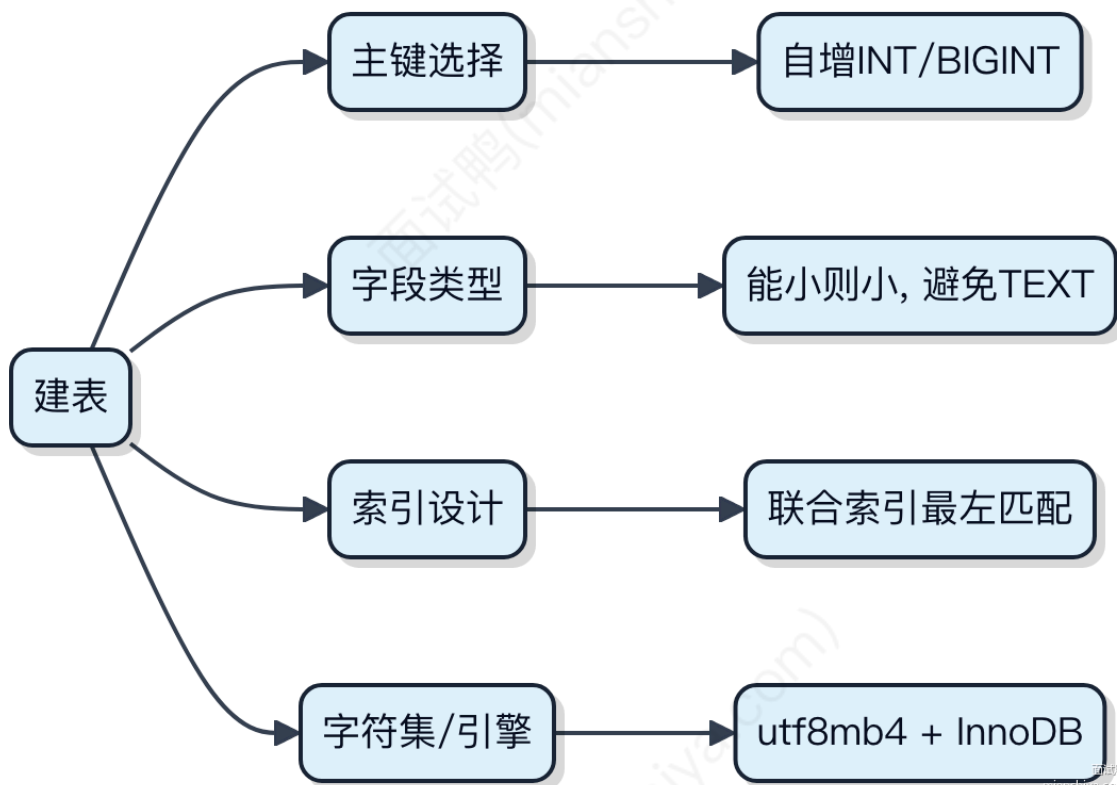
主键设计一般用自增 ID，别用 UUID 这种长字符串，索引膨胀快，写入也慢。InnoDB 是索引组织表，主键直接决定数据物理存储顺序，主键越小，辅助索引也越小。

字段类型能小则小。比如状态字段用 `tinyint` 就够了，别上 `int`。时间用 `datetime` 或 `timestamp`，后者还支持时区，占用 4 字节。 `text` 能不用就不用，大字段会触发行溢出，影响查询效率。

索引不是越多越好。每加一个索引都得维护 B+ 树，写入变慢。联合索引注意最左前缀原则，比如 (a,b,c) 能命中 a、a,b、a,b,c 的查询，但 b,c 压根不经过这个索引。

- 1) 避免使用 SELECT *，只查需要的字段，减少回表和网络开销
- 2) 给经常用于查询、排序、分组的字段加索引，但单表索引建议不超过 6 个
- 3) 大表做 ALTER TABLE 要小心，生产环境可能锁表，可以用 pt-online-schema-change 工具在线改

字符集统一用 utf8mb4，支持 emoji 没毛病。别忘了设置合适的存储引擎，现在基本都是 InnoDB，支持事务和行锁。



为什么在 MySQL 中不推荐使用多表 JOIN?

MySQL 里不推荐多表 JOIN，主要是它容易把简单问题搞复杂，尤其在数据量上来以后，数据库的负担会明显加重。

- 1) 查询优化器对 JOIN 的执行计划可能不够理想。比如五张表 JOIN，如果关联字段没走好索引，或者统计信息不准，就可能导致笛卡尔积或全表扫描，响应时间从毫秒飙到几秒。
- 2) JOIN 操作通常在数据库层完成，意味着所有中间结果要在内存里处理。假设两张表各 100 万行，连接后临时结果集可能达到上百万行，这会吃掉大量 Buffer Pool，影响其他查询。
- 3) 横向扩展困难。应用层可以加机器水平扩容，但数据库做分布式 JOIN 成本很高。像 ShardingSphere 这类中间件虽然支持，但跨库 JOIN 性能差，一般建议业务自己拆解。

更常见的做法是：把多表 JOIN 拆成多个单表查询，在应用层拼装数据。比如先查订单主表拿到 user_id，再用 IN 批量查用户表。虽然多了几次网络请求，但每个查询都能走索引，整体更可控。



当然也不是完全不能用 JOIN。两三个表、数据量不大、有合适索引的情况下，一条 SQL 查完反而是最高效的。关键是要清楚它的代价，别让数据库干太多脏活累活。

MySQL 的查询优化器如何选择执行计划？

MySQL 拿到一条 SQL，不会直接去查数据，而是先走查询优化器生成执行计划。选哪个索引、要不要用索引合并、走不走覆盖索引，这些都由它决定。

优化器不是瞎猜的，它基于**成本模型**做决策。会估算每种可能的执行路径代价，比如扫描多少行、随机 IO 次数、是否需要排序或临时表，最后挑个总成本最低的方案。

- 1) 首先看 WHERE 条件里的字段有没有可用索引，通过统计信息判断索引区分度。比如 `WHERE user_id = 100`，如果这个字段有索引且数据分布均匀，大概率会走索引。
- 2) 多条件查询时可能考虑索引合并 (index merge)，但实际效果不一定好，有时候全表扫描反而更快。
- 3) 关联查询的顺序也会被重排。比如 `A JOIN B ON ...`，优化器可能把小结果集的表放前面，减少循环次数。像从 `orders` 和 `order_items` 关联查数据，它可能会先过滤出少量订单再关联明细。

```
-- 优化器可能改写执行顺序，不按 SQL 书写顺序
SELECT * FROM orders o JOIN order_items i ON o.id = i.order_id
WHERE o.status = 'paid';
```

- 4) 用 `EXPLAIN` 能看到最终选定的执行计划，重点关注 type (访问类型)、key (用了哪个索引)、rows (扫描行数)、Extra (是否 Using filesort)。

统计信息不准会导致误判，比如刚大批量导入数据后，可以手动 `ANALYZE TABLE` 更新。



如何实现数据库的不停服迁移？

不停服迁移的核心是**双写同步**和**数据校验**，整个过程要保证业务无感、数据不丢。

- 1) 先做结构兼容改造。老库和新库字段设计得能共存，比如新增非空字段要有默认值，避免双写时报错。
- 2) 开启双写。在代码里把数据库操作改成同时写两份，老库和新库都落数据。读还是只从老库读，这时候新库只是个“影子库”。

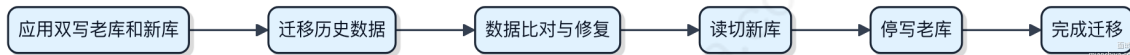
```
// 伪代码示意：双写阶段
userDao.writeToOldDB(user);
userDao.writeToNewDB(user); // 异步发消息也可以，但要考虑失败重试
```

- 3) 跑数据迁移工具，把老库的历史数据搬过去。可以用 DataX 或 Canal + Kafka + Flink 这类组合，边搬边追增量。
- 4) 数据比对。拿工具抽样比对两边的数据一致性，比如阿里的 DataDiff，发现不一致就补。

5) 切换读流量。确认数据基本一致后，先把读请求切到新库，观察一段时间。

6) 下线双写。读稳定了，再把双写关掉，只写新库，老库归档。

整个过程最怕的是中间出错导致数据不一致。所以每一步都要有监控和回滚方案，比如用 **Canal** 监听 binlog 做实时反向补偿。



MySQL 数据库的性能优化方法有哪些？

数据库性能优化得从底层原理出发，不能只堆索引。先说最关键的几个点。

索引失效是高频坑，比如在 where 条件里对字段做函数操作，`WHERE YEAR(create_time) = 2023`，这索引就压根不经过。联合索引要遵守最左前缀原则，中间断了后面部分就用不上。像 `idx(a,b,c)`，查询条件只有 a 和 c，那 c 的部分就走不了索引。

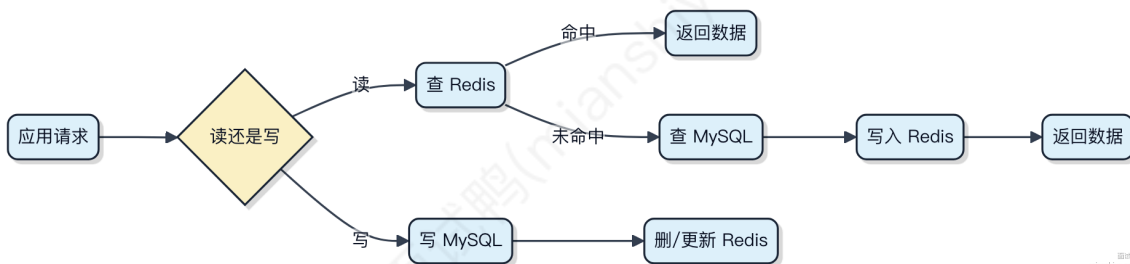
别忘了覆盖索引。如果查询的字段都在索引里，压根不用回表，性能提升明显。比如 `SELECT id FROM user WHERE status=1`，id 是主键，status 有索引，直接走索引返回结果。

SQL 慢查得靠分析。EXPLAIN 看执行计划，重点关注 type（最好到 range 以上）、key（走了哪个索引）、rows（扫描行数）。超过 1 秒的查询就得警惕，线上一般要求控制在 50ms 内。

大表分页用游标分页替代 `LIMIT offset, size`。比如按时间排序，记录上次查询的最大时间戳，下一页从这个时间之后查，避免深翻页全表扫描。

读多写少场景可以用缓存。Redis 缓存热点数据，减少数据库压力。但要注意缓存穿透、雪崩问题，加布隆过滤器或者设置空值缓存。

冷热数据分离也常见。用户登录日志这种，写完基本不读，可以定期归档到 Hive 或 Tidb，主库只留最近 3 个月数据。



最后，连接池配置合理，HikariCP 一般把最大连接数设为 20-50，太多反而会拖垮数据库。

MySQL 三层 B+ 树能存多少数据？

MySQL 的 InnoDB 引擎用 B+ 树组织数据，叶子节点存的是完整行记录，非叶子节点存的是索引键和指针。我们通常说的“三层 B+ 树”指的是根节点、中间层、叶子层。

先算单页容量。InnoDB 默认页大小 16KB。假设主键是 bigint，占 8 字节，指针一般按 6 字节算，那一个页能存的索引项数大约是：

每项占用 $8 + 6 = 14$ 字节

一页可存约 $16 * 1024 / 14 \approx 1170$ 个索引项

也就是说，一层非叶子节点最多指向 1170 个下级页。

再看叶子节点。假设一行数据平均 1KB，那一个 16KB 的页能存 16 行数据。

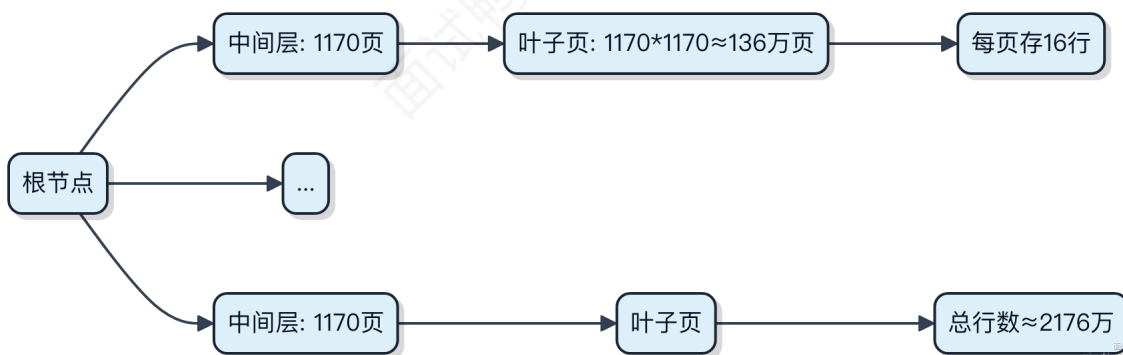
那么三层结构的数据容量就是：

根节点 → 1170 个中间页 → 每个中间页再指向 1170 个叶子页 → 总共约 $1170 * 1170 \approx 136$ 万个叶子页

每个叶子页存 16 行 → 总容量约 $136 万 * 16 \approx 2176$ 万行

所以，一个典型的三层 B+ 树能存 **两千万级别** 的数据行。

如果数据量超过这个规模，B+ 树会变成四层，查询就需要多一次磁盘 IO。从内存访问到落到磁盘，延迟可能从几十纳秒跳到几毫秒，性能差距巨大。

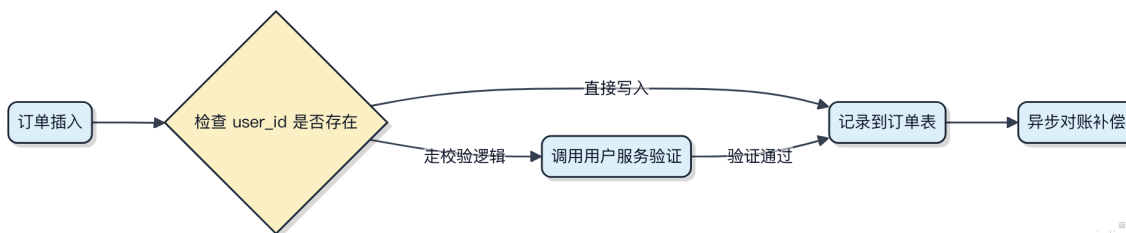


什么是数据库的逻辑外键？数据库的物理外键和逻辑外键各有什么优缺点？

物理外键是数据库层面通过 DDL 定义的约束，比如在 MySQL 中用 `FOREIGN KEY` 显式声明两个表之间的关联关系。数据库会在增删改时自动校验引用完整性，**数据一致性**有强保障。

逻辑外键则是没有实际 DDL 约束，只靠应用代码或约定来维护表之间的关联关系。比如订单表里有个 `user_id` 字段指向用户表主键，但没建外键约束，全靠程序保证不出现无效用户 ID。

- 1) 物理外键的优点在于数据库能自动阻止非法数据写入，级联操作也方便，像 `ON DELETE CASCADE` 可以自动清理子记录。缺点是会带来锁竞争，在高并发场景下容易成为瓶颈，分库分表后也难以跨节点生效。
- 2) 逻辑外键的好处是灵活，适合分布式架构，比如使用 ShardingSphere 或 MyCAT 时，外键没法跨片存在。应用层可以自己控制校验时机和重试策略。但坏处也很明显，一旦代码出错或者新同事不了解约定，很容易产生脏数据。
- 3) 实际项目中，单体数据库一般建议用物理外键，省心又安全。微服务+分库分表场景下基本都用逻辑外键，毕竟分布式外键代价太大，不如交给服务层统一处理，配合定时对账任务来兜底。



要不要用物理外键，关键看你的系统能不能接受它带来的耦合和性能开销。

MySQL 事务的二阶段提交是什么？

MySQL 的二阶段提交（2PC）是 InnoDB 在崩溃恢复时保证事务持久性和数据一致性的关键机制，主要解决 redo log 和 binlog 的一致性问题。

redo log 记录物理日志，确保事务的原子性和持久性；binlog 是逻辑日志，用于主从复制和 PITR（时间点恢复）。如果两者提交不同步，崩溃后可能不一致。比如事务提交了但 binlog 没写，备库就会丢数据。

为了解决这个问题，InnoDB 采用二阶段提交：

- 1) prepare 阶段：事务产生的 redo log 写入磁盘，并将事务状态标记为 PREPARED。此时 binlog 还没落盘。
- 2) commit 阶段：把 binlog 写入并刷盘，然后 InnoDB 再写一条 commit 标记到 redo log。只有这一步完成，事务才算真正提交。

崩溃恢复时，MySQL 会扫描 undo log 中的 PREPARED 事务，拿着事务的 XID 去 binlog 里查是否存在。如果 binlog 里有这个 XID，说明它已经完成了 prepare，就应该被提交；否则回滚。



这个机制让主从之间的数据同步更可靠，也是 MGR、XtraDB Cluster 等高可用方案的基础。但注意，二阶段提交只在开启 binlog 且使用支持事务的引擎时才生效。

MySQL 的乐观锁和悲观锁是什么？

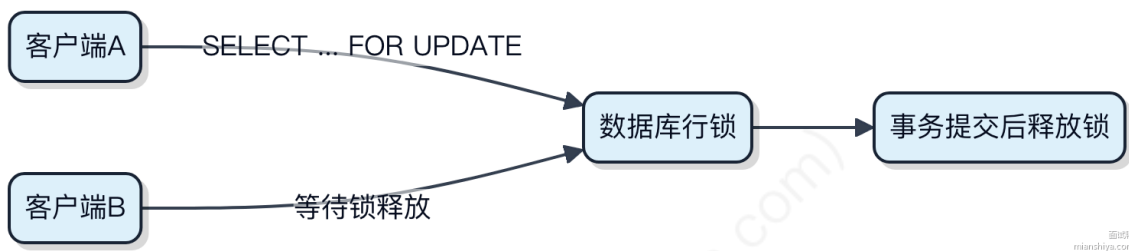
数据库里的锁机制，其实是为了处理并发场景下数据一致性问题的两种思路。乐观锁和悲观锁不是 MySQL 数据库层直接提供的锁类型，而是应用层面的一种设计思想，不过可以用数据库的能力去实现。

- 1) 悲观锁认为冲突大概率会发生，所以访问数据时就直接加锁。在 MySQL 里通常用 `SELECT ... FOR UPDATE` 或 `SELECT ... LOCK IN SHARE MODE` 实现。比如扣减库存这种敏感操作，你查一行记录马上加排他锁，别的事务就只能等着，直到当前事务提交释放锁。这种方式能保证强一致性，但吞吐量会受影响，特别是高并发下容易阻塞。
- 2) 乐观锁相反，它假设冲突很少发生，不会提前加锁。一般通过版本号或时间戳字段来控制。比如读取数据时带一个 version 字段，更新时判断这个 version 是否被别人改过：

```
UPDATE goods SET stock = stock - 1, version = version + 1
WHERE id = 1001 AND version = 1;
```

如果影响行数为 0，说明 version 不匹配，更新失败，需要重试。这种方式没有长时间持有锁，适合读多写少的场景，像秒杀系统里常用这种机制配合 Redis 预减库存来扛住流量。

- 3) 对比来看，悲观锁简单直接，但容易搞成串行执行；乐观锁并发性能好，但会有失败重试的成本。选哪种得看业务场景。比如金融转账偏向悲观锁保安全，内容点赞这类弱一致性操作更适合乐观锁。

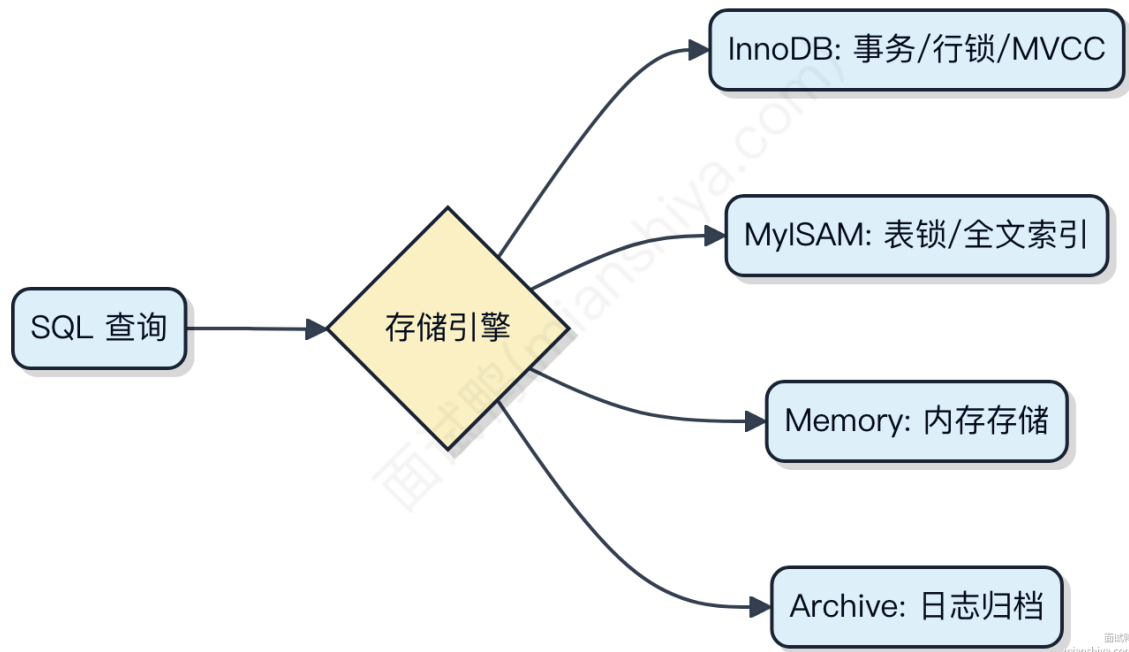


MySQL 的存储引擎有哪些？它们之间有什么区别？

MySQL 支持多种存储引擎，但最常用的其实是 InnoDB 和 MyISAM。面试里问这个，核心是看你对 **InnoDB** 的掌握程度。

- 1) InnoDB 是 MySQL 默认引擎，从 5.5 版本开始就扛住主要业务场景。它支持事务、行级锁、外键约束，还实现了 MVCC 来提升并发性能。大多数高并发、要求数据一致性的系统都会选它，比如电商订单、支付这类模块。
- 2) MyISAM 虽然查询快，尤其是 `count(*)` 这种全表扫描，但它压根不支持事务和行锁，写入时会锁整张表。现在基本被淘汰了，只在一些日志类、读多写少的场景还能见到，比如老系统的数据分析表。
- 3) 其他引擎像 Memory 是把数据放内存的，断电就丢，适合做临时缓存表；Archive 用来存归档数据，压缩比高，但查询能力弱。

InnoDB 的数据是按聚簇索引组织的，主键索引直接带行数据，二级索引指向主键。而 MyISAM 不管什么索引，都指向数据文件地址，结构上就简单很多。



选引擎的关键是看业务需求。要事务和并发写，必须用 InnoDB；纯读且统计频繁，可以考虑 MyISAM，但现在一般通过缓存或数仓解决。别在新项目里用 MyISAM，搞不定故障恢复。

MySQL InnoDB 引擎中的聚簇索引和非聚簇索引有什么区别？

聚簇索引的叶子节点直接存储完整的行数据，而非聚簇索引存的是索引列值和主键ID。InnoDB 表必须有**聚簇索引**，一般是主键，如果没有主键会选唯一非空索引，再没有就隐式生成一个。

查询时，走聚簇索引可以直接拿到数据，一次查找就行。走非聚簇索引需要先查到主键ID，再拿主键去聚簇索引回表查数据，多了一次IO。

比如你用 `SELECT * FROM users WHERE id = 1`，id 是主键，直接走聚簇索引搞定。但用 `SELECT * FROM users WHERE name = 'Tom'`，name 上有普通索引，就得先查 name 索引得到 id，再去主键索引查整行，这叫回表。

非聚簇索引的结构就像一棵B+树，内部节点只存 name 和指针，叶子存 name + id。聚簇索引的B+树叶子才真正放整行记录。

更新主键成本很高，因为聚簇索引里数据是按主键物理排序的，改主键等于挪动数据页位置。所以主键建议用自增int或bigint，避免页分裂。

- 1) 聚簇索引决定数据存储顺序
- 2) 每个表只有一个聚簇索引
- 3) 非聚簇索引依赖主键做“跳转”



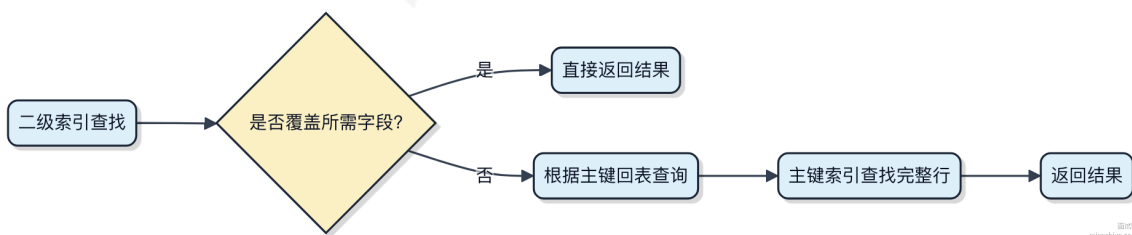
别在频繁更新的列上建主键，也别让主键太长，像用UUID那种，一页存不下几条数据，B+树层数变深，查询就慢了。

MySQL 中的回表是什么？

回表指的是当查询不能完全通过索引树获取所需数据，必须回到主键索引（聚簇索引）中去查找剩余字段的过程。InnoDB 的二级索引叶子节点存的是主键值，不是行数据地址，所以一旦需要非索引列，就得拿主键再去查一遍主键索引。

比如你有个联合索引 (name)，但你要查 name 和 email，而 email 不在索引里，这时就会先在 name 索引树上找到主键 ID，再拿着 ID 去主键索引上找完整的行记录，这就是一次回表。

- 1) 回表的代价其实不低，一次查询可能变成两次 B+ 树遍历，尤其是结果集大时，随机 IO 会明显增加。
- 2) 避免回表的关键是让查询能走**覆盖索引**，也就是所有要查的字段都在索引里。像 `SELECT id, name FROM users WHERE name = 'Tom'` 就不需要回表，因为 id 和 name 都在索引中。
- 3) 使用联合索引或调整索引结构可以有效减少回表。例如把 (name, email) 建成联合索引，查这两个字段就直接命中，压根不经过主键索引。



简单说，回表就是“查了索引还得再查数据行”，能覆盖就不回，不能覆盖就得来回跑。

MySQL 的覆盖索引是什么？

覆盖索引指的是，查询的字段都包含在某个索引中，MySQL 直接通过索引就能返回数据，压根不经过主键索引的回表操作。

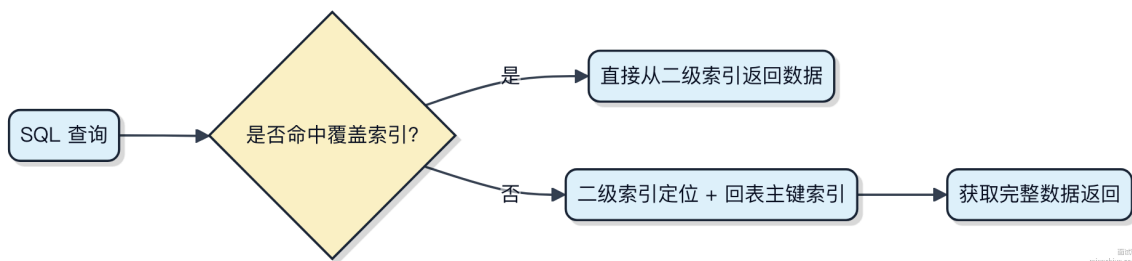
比如有张订单表 orders，建了联合索引 (status, create_time, user_id)。如果执行这样的查询：

```
SELECT user_id FROM orders WHERE status = 1 AND create_time > '2023-01-01';
```

它只需要走索引树查找，找到对应索引项后直接取出 user_id 返回，不需要再去主键索引捞整行数据。这种情况下，这个索引就起到了覆盖索引的作用。

有几个关键点要注意：1) 如果查询里有个字段没在索引里，比如 SELECT amount，那肯定得回表。2) 覆盖索引是提升性能的利器，尤其在大表上能减少大量随机 IO。3) 但别为了覆盖所有查询建宽索引，索引太宽写入代价高，B+ 树层数可能变多，反而影响整体性能。

实际场景中，像用 Elasticsearch 同步 MySQL 数据做搜索时，常会为高频查询字段建覆盖索引，让 MySQL 只负责写和增量同步，查都交给 ES，这时候覆盖索引能让同步查询轻量化。



MySQL 的索引下推是什么？

索引下推（ICP）是 MySQL 5.6 引入的优化机制，主要解决二级索引查询时回表次数过多的问题。没有 ICP 时，存储引擎找到一条符合索引条件的记录，就会立刻回表查聚簇索引，再由 Server 层判断其他 WHERE 条件是否满足。如果筛选条件多，很多回表其实是白做的。

有了 ICP，存储引擎会把能用索引字段的过滤条件也带上，在索引遍历过程中就提前过滤掉不满足条件的数据，**只把真正符合条件的才回表**。这样大大减少了回表次数，尤其在联合索引包含多个筛选字段时效果明显。

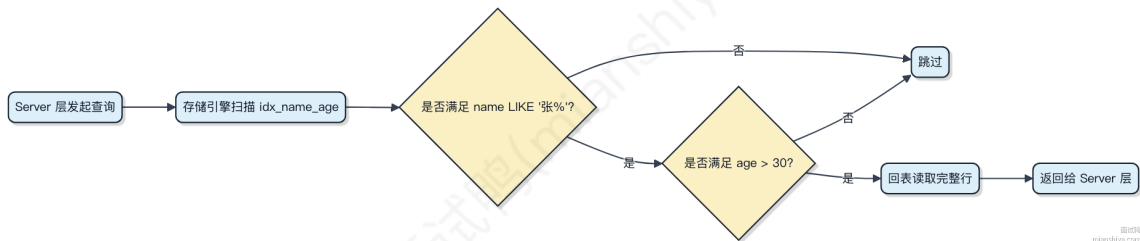
比如这张表：

```
CREATE TABLE users (
  id int primary key,
  name varchar(50),
  age int,
  city varchar(20)
);
-- 建立联合索引
ALTER TABLE users ADD INDEX idx_name_age (name, age);
```

执行这条查询：

```
SELECT * FROM users
WHERE name LIKE '张%'
AND age > 30;
```

name 是范围查询，传统方式只能用到 name 的索引部分，age 的条件得回表后判断。但启用 ICP 后，age > 30 这个条件会被“下推”到存储引擎层，在遍历 idx_name_age 索引时直接跳过 age <= 30 的记录，减少一半以上的回表量。



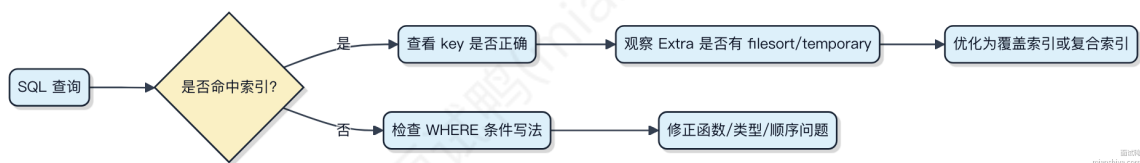
注意 ICP 不适用于主键索引，也不支持覆盖索引的情况——因为压根不经过回表。一般像 InnoDB、MyISAM 都支持，但具体还得看执行计划里 Extra 字段有没有 Using index condition。

MySQL 中使用索引一定有效吗？如何排查索引效果？

索引不是银弹，用不对压根不生效。常见原因比如写法导致无法命中，像在索引字段上做函数操作 WHERE YEAR(create_time) = 2023，这直接让 B+ 树失效。或者隐式类型转换，比如索引字段是 varchar，但用数字查询没加引号，MySQL 会全表扫描。

联合索引要遵守最左前缀原则，中间跳字段或者顺序颠倒都可能让索引失效。比如索引 (a,b,c)，查 b=1 and c=2 就用不上。还有 like 以 % 开头，LIKE '%abc' 这种也无法利用索引查找。

想确认索引有没有起作用，核心是看执行计划。用 EXPLAIN 分析 SQL，重点关注 type、key 和 Extra 字段。type 至少到 range 级别才算有效使用，最好能达到 ref 或 const。key 显示实际走的索引，如果是 NULL 就说明没用上。Extra 里出现 Using index 表示覆盖索引命中，性能好；要是出现 Using filesort 或 Using temporary，那多半有优化空间。



线上排查可以用 slow query log 配合 pt-query-digest 分析慢查询，定位哪些 SQL 没走索引。有时候加了索引但数据量小，优化器觉得全表扫更快，也会放弃索引，这种情况一般不用管。

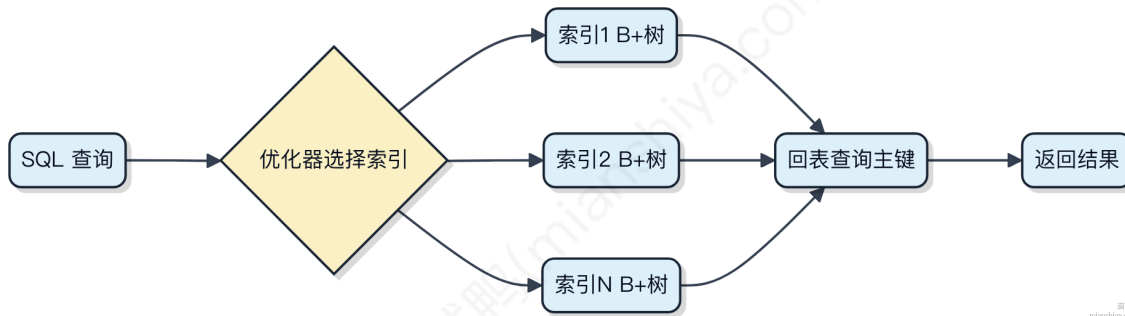
MySQL 中的索引数量是否越多越好？为什么？

索引不是越多越好，它是一把典型的双刃剑。你加一个索引，查询可能快了 10 毫秒，但写入就得付出额外代价。

每次执行 INSERT、UPDATE 或 DELETE，MySQL 都得去维护对应的 B+ 树索引。每多一个索引，就意味着多一次磁盘 I/O 和内存操作。比如一张用户表，如果你给 name、email、phone、age、city 各建一个单列索引，那每次插入一条数据，引擎最多得更新 5 棵 B+ 树。这开销在高并发写入场景下会直接拖垮性能。

另外，索引占用存储空间。每个索引都对应物理上的数据结构，尤其是组合索引列多的时候，可能比数据本身还大。更麻烦的是优化器选错执行计划——索引太多会让优化器计算成本时耗时增加，甚至选中不合适的索引，导致慢查询。

一般建议控制单表索引数量在 5 个以内，优先使用覆盖索引和组合索引。比如你经常查 city 和 age，就建 (city, age) 而不是两个单列索引。这样既能命中联合查询，又能减少维护成本。



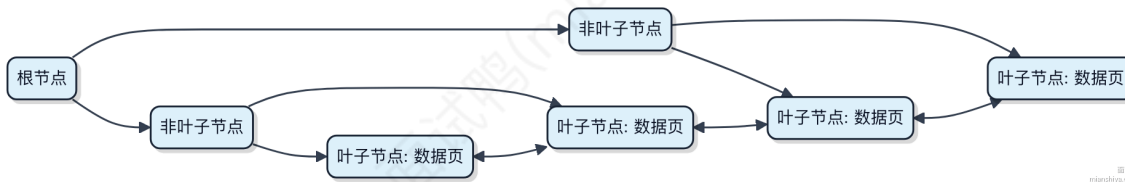
重点是权衡读写比例。读多写少的分析表可以适当多建索引；频繁写入的交易流水表，就得精打细算。

为什么 MySQL 选择使用 B+ 树作为索引结构？

MySQL 的索引选型，关键是要在磁盘 I/O 效率、数据有序性和范围查询能力之间找到平衡。B+ 树在这几个方面表现非常均衡。

- 1) B+ 树的非叶子节点不存数据，只存索引，这意味着同样大小的页可以容纳更多键值，树的高度更低。一般 3 层的 B+ 树就能支撑上百万行的数据查询，查一次数据最多 2~3 次磁盘 IO，效率很高。
- 2) 所有数据都集中在叶子节点，并且叶子节点之间用双向链表连接，这使得范围查询特别高效。比如 `WHERE id BETWEEN 100 AND 200`，只需要定位到第一个 key，然后顺序往后扫就行，不需要回溯父节点。
- 3) B+ 树的结构稳定，插入删除时通过分裂和合并维持平衡，不会频繁重构整棵树。像 MyISAM 和 InnoDB 都基于 B+ 树实现主键和二级索引，InnoDB 的聚簇索引甚至把主键和行数据直接绑在一起，减少一次回表。

对比一下，哈希索引适合等值查询，但压根不支持范围查询；二叉搜索树虽然快但深度太深，IO 次数多；B 树每个节点都存数据，导致单层能放的 key 变少，树更高。



这种设计让 MySQL 能扛住大量并发查询和写入，尤其是业务系统里常见的分页、排序、区间统计场景。

MySQL 中长事务可能会导致哪些问题？

长时间运行的事务在 MySQL 里是个隐患，尤其在高并发场景下容易引发连锁反应。

1) 锁竞争加剧

长事务持有锁的时间更久，别的事务等锁的时间变长，响应变慢。比如一个事务一直没提交，它锁住的行别人就改不了，严重时直接把线程池拖垮，大量连接堆积。

2) 回滚段压力大

InnoDB 的 MVCC 依赖 undo log 维护多版本数据。长事务不结束，undo log 就不能清理，这些历史版本一直占着空间。极端情况下，可能让 ibdata1 文件疯狂增长，磁盘撑爆。

3) 主从延迟

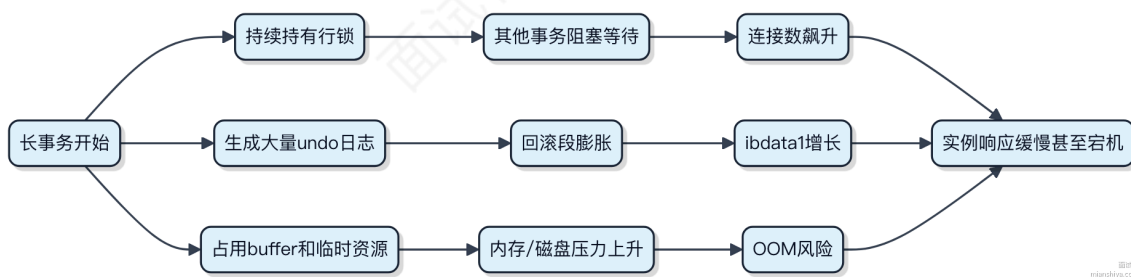
主库上执行了 5 分钟的事务，从库得按同样的节奏重放。如果这个事务还涉及大量行变更，从库 SQL Thread 跟不上节奏，延迟就来了。像阿里云 RDS 这类环境，延迟超过 30 秒可能触发告警。

4) 临时文件和内存泄漏风险

事务过程中产生的临时结果集、排序缓冲都得维持状态。时间越长，内存占用越高，可能触发 OOM。特别是 ORDER BY、GROUP BY 操作，容易用到磁盘临时表。

5) 逻辑备份失败

mysqldump 默认用 REPEATABLE READ 隔离级别加全局读锁，长事务会让 FLUSH TABLES WITH READ LOCK 卡住，导致备份超时。换成 --single-transaction 也一样，如果备份期间有长事务，快照一致性就保不住。



一般建议控制在几秒内完成，超过 30 秒就算危险信号。可以查 `information_schema.innodb_trx` 表定位异常事务。

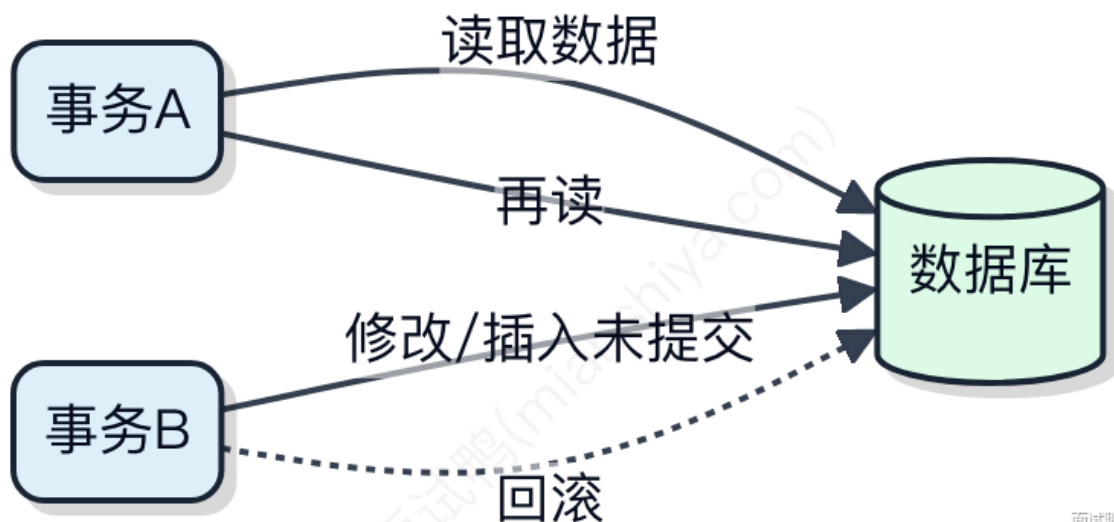
数据库的脏读、不可重复读和幻读分别是什么？

脏读、不可重复读、幻读，都是在并发事务下读取数据时可能遇到的一致性问题。它们的根源在于事务之间没有做好隔离。

1) 脏读是指一个事务读到了另一个事务未提交的数据。比如事务 A 更新了一行记录但还没提交，事务 B 此时读到了这行“临时”数据。如果事务 A 最终回滚，那事务 B 读到的就是无效数据。这种情况在金融转账场景里绝对不能接受。

2) 不可重复读指的是在同一事务中，两次读取同一行数据结果不一样。原因是另一个事务在这期间修改并提交这行数据。例如事务 A 先读某条记录 age=20，事务 B 修改为 25 并提交，事务 A 再读一次发现 age 变了。这会影响需要多次校验一致性的逻辑。

3) 幻读和不可重复读类似，但针对的是数量变化。比如事务 A 查询 id > 10 的记录有 5 条，事务 B 插入一条新的满足条件的记录并提交，事务 A 再查发现变成 6 条，像“幻觉”一样多出一条。典型的场景是分页查询时前后翻页数据错乱。



面试鸭
mianshiya.com

这三个问题可以通过数据库的事务隔离级别来控制。读已提交能解决脏读，可重复读能解决前两个，串行化才能彻底避免幻读。MySQL InnoDB 在可重复读级别下通过 **间隙锁** 和 MVCC 机制也能防止大部分幻读场景。

MySQL 中有哪些锁类型？

MySQL 的锁机制主要是为了解决并发访问带来的数据不一致问题。不同场景下，引擎会选择不同的锁来平衡性能和一致性。

先说 InnoDB 支持的锁类型。最常见的是**行级锁**，它能最大程度减少锁冲突，提升并发能力。行级锁分两种：共享锁（S锁）和排他锁（X锁）。事务对某行加了 S 锁，其他事务还能加 S 锁读，但不能加 X 锁写；加了 X 锁，其他事务啥都不能干。

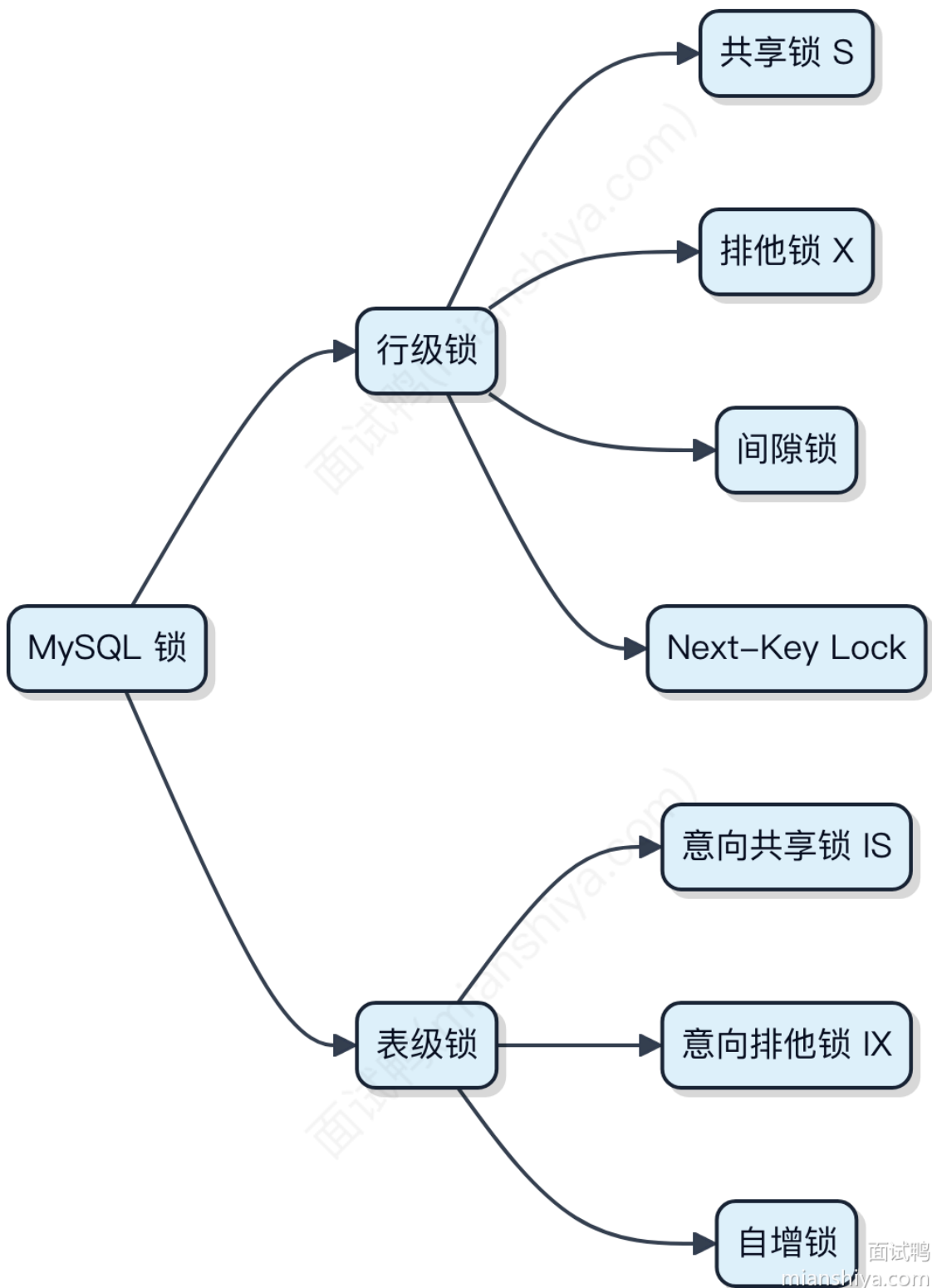
除了行锁，还有**间隙锁**（Gap Lock），它不锁记录本身，而是锁索引之间的“间隙”，防止幻读。比如你查 `WHERE id > 10 AND id < 20`，InnoDB 会锁住 (10,20) 这个区间，不让别人插入新记录。

Next-Key Lock 是行锁和间隙锁的组合，既锁记录又锁间隙，是可重复读隔离级别下防幻读的关键。RR 级别默认用的就是这种锁。

表级锁也有，比如意向共享锁（IS）和意向排他锁（IX）。它们是“声明式”锁，表示事务打算在某个表的某些行上加 S 或 X 锁。有了意向锁，就不需要逐行检查就能快速判断表是否被锁定。

自增锁是特殊的表级锁，用在有 AUTO_INCREMENT 字段的表上，保证自增值分配不冲突，INSERT 完就释放。

- 1) 行级锁：精确控制，并发高
- 2) 间隙锁：锁范围，防幻读
- 3) Next-Key Lock：行锁 + 间隙锁，RR 默认
- 4) 表级锁：意向锁、自增锁等，协调行锁与表操作



[为什么阿里巴巴的 Java 手册不推荐使用存储过程？](#)

阿里不推荐用存储过程，主要不是技术行不行的问题，而是工程协作和系统演进的现实考量。

- 1) 部署和版本管理太麻烦。存储过程写在数据库里，改一次要单独发脚本，跟应用代码的 Git 流程脱节。上线容易漏，回滚也难搞，CI/CD 流水线直接就断了。
- 2) 调试和测试成本高。你没法像 Java 代码那样用 IDE 单步调试，得靠数据库工具打日志、查 trace。单元测试基本靠手敲 SQL，自动化难做，覆盖率压根没法保证。
- 3) 可移植性差。MySQL 的存储过程换到 Oracle 上基本重写，函数、语法、游标处理全不一样。一旦考虑异构数据库迁移或者分库场景，这类代码就是包袱。
- 4) 把业务逻辑塞进数据库，等于让 DB 承担不该它干的脏活累活。数据库应该专注数据持久化、事务、索引这些事，扛住 QPS 和连接数压力。业务规则放在应用层，用 Java 写，可读性、扩展性、团队协作都更可控。

当然，特定场景比如金融批处理、报表聚合，对实时性要求低、计算密集的，用存储过程或 PL/SQL 还是能省网络开销。但这类需求在互联网架构里占比很小，一般用 Spark 或 Flink 更合适。

所以手册的建议是基于大规模团队协作和敏捷交付的权衡，不是说存储过程本身有原罪。

MySQL 中如果发生死锁应该如何解决？

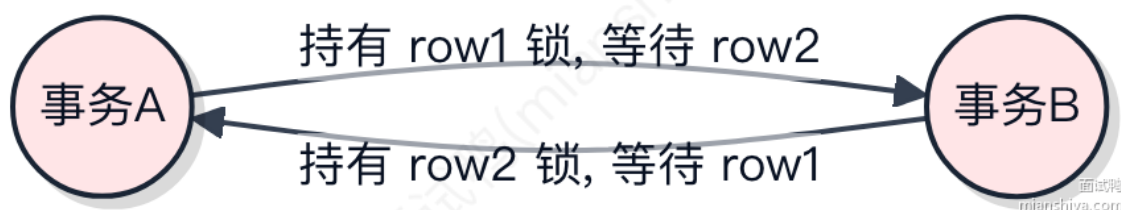
死锁是多个事务循环等待对方持有的锁，谁都继续不了。MySQL 会自动检测到这种情况，在几秒内主动回滚其中一个事务，让另一个先走，避免一直卡着。

被回滚的事务会收到一个错误提示，比如 `Deadlock found when trying to get lock`。这时候应用层得捕获这个异常，一般做法是做几次重试，大概率就能成功了。别忘了加个退避策略，别上来就疯狂重试。

想看看到底怎么死的，可以查 `information_schema.innodb_lock_waits` 和 `innodb_trx` 这两表，结合 `SHOW ENGINE INNODB STATUS` 输出的最近一次死锁详情。这里面会告诉你哪个事务持了什么锁，等谁，谁又在等它，链条一清二楚。

避免死锁的关键是访问多张表或同一张表多行时，**顺序一致**。比如都按主键从小到大更新，或者固定先操作用户表再订单表。乱序操作很容易撞上“你等我、我等你”的局面。

代码里注意缩短事务长度，别把网络请求、文件读写这些慢操作包在事务里。长事务持有锁的时间越长，撞车概率直线上升。



其实大多数死锁不是 MySQL 的问题，而是业务逻辑没设计好。只要保证加锁顺序一致，事务够短，重试机制到位，系统基本能扛住。

如果组长要求你主导项目中的分库分表，大致的实施流程是？

先别急着上工具，分库分表是个系统工程，搞不好反而增加运维成本还搞不定问题。第一步得把现状摸清楚，当前的数据量级是多少，比如单表数据已经到了 5000 万行以上，查询明显变慢，TPS 掉到 200 以下，这才具备拆分的前提。

- 1) 明确拆分目标。是读多写少？还是写入瓶颈？根据业务场景选合适的拆分策略。比如订单系统按 `user_id` 分片，日志类数据可以按时间分片。这里的关键是选择**分片键**，它直接影响后续的查询路由和数据倾斜问题。

- 2) 技术选型要贴合现有架构。如果用的是 Java 技术栈, ShardingSphere 是主流选择, 支持 JDBC 和 Proxy 两种模式。JDBC 适合应用内嵌, Proxy 更适合多语言场景。配置好分片规则后, SQL 解析引擎会自动将请求路由到对应库表。
- 3) 数据迁移和双写同步不能少。上线前通过离线任务把历史数据按新规则打散, 同时改造服务层支持双写, 确保过渡期数据不丢。可以用 Canal 监听原库 binlog 做增量补偿。
- 4) 灰度上线+流量回放。先切少量流量验证, 用真实请求压测分片逻辑是否正确, 监控各节点负载是否均衡。



MySQL 中 DELETE、DROP 和 TRUNCATE 的区别是什么?

这三个命令看着都是删数据的, 其实差别挺大的, 用错地方可能出大问题。

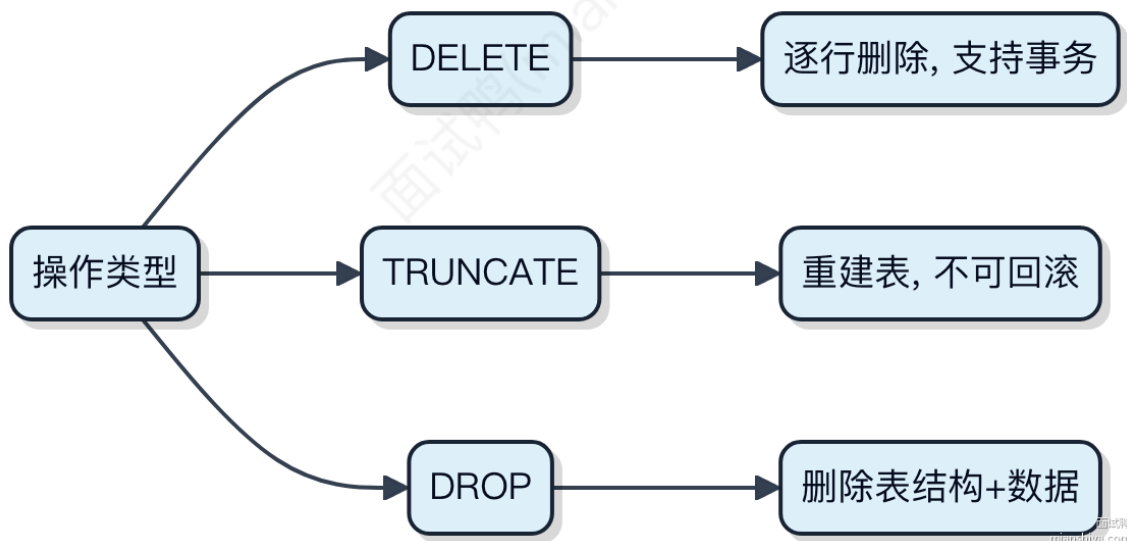
DELETE 是一条一条把数据从聚簇索引里删掉, 每删一行都会记录 undo log 和 redo log, 支持回滚。你可以带 WHERE 条件删部分数据, 比如 `DELETE FROM users WHERE id < 100`。执行完不会释放表空间, 自增 ID 也不重置。InnoDB 里这操作还容易锁行, 删几万条可能就把表搞慢了。

TRUNCATE 本质是 DDL 操作, 它直接把整个表 drop 掉再重建, 速度非常快。不能加 WHERE 条件, 只能清空整张表。执行后自增 ID 会归 1, 空间也会被系统回收。因为不逐行删除, 所以不走事务, 不能回滚。权限要求也更高。

DROP 就更狠了, 不只是删数据, 连表结构一起干掉。索引、约束、触发器全都没了, 数据文件直接删除。想恢复基本只能靠备份。比如 `DROP TABLE users` 执行完, 这张表在数据库里就彻底消失了。

- 1) DELETE 支持事务和条件删除, 适合删少量数据
- 2) TRUNCATE 清空整表快, 但不可回滚, 自增重置
- 3) DROP 连表带数据全删, 恢复成本最高

举个场景: 你清理日志表, 每天删百万条, 用 DELETE 可能跑几小时, 换成 TRUNCATE 几秒完事。但如果只是下架某个用户, 那必须用 DELETE 带条件删。



如何在 MySQL 中监控和优化慢 SQL?

开启慢查询日志是第一步，MySQL 会把执行时间超过 `long_query_time` 的语句记录下来。默认这个阈值是 10 秒，但生产环境一般设成 1 秒甚至 500 毫秒。记得打开 `slow_query_log` 开关，不然日志压根不生成。

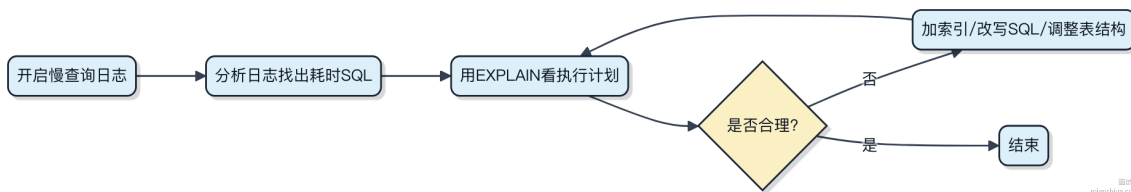
拿到慢 SQL 后，`EXPLAIN` 是你的主要武器。看它的执行计划，重点关注是否走了索引、扫描行数（rows）有多大、有没有用临时表或文件排序。type 显示 ALL 就是全表扫描，通常要优化。

- 1) 先查索引情况，`SHOW INDEX FROM table_name` 看关键字段有没有有效索引
- 2) 复合索引注意最左匹配原则，中间断了就用不上
- 3) `SELECT *` 能少用就少用，只取需要的字段，避免回表

真正复杂的场景可以用 **Performance Schema** 或 **sys schema**，比如查

`sys.statements_with_full_table_scans` 直接找出全表扫描的语句。阿里开源的 Arthas 也能动态抓 SQL 执行情况。

代码层面别写 N+1 查询，循环里发 SQL 数据库肯定扛不住。ORM 框架像 MyBatis 要警惕自动拼装的 SQL，必要时手写。



线上改表用 `pt-online-schema-change` 这类工具，避免锁表。优化不是一锤子买卖，得持续盯着，尤其是业务高峰期。

MySQL 中如何解决深度分页的问题?

深度分页在 MySQL 里是个经典性能问题，尤其是用 `LIMIT offset, size` 这种方式，当 offset 很大时，比如 `LIMIT 1000000, 20`，数据库得先扫描前 100 万条数据，再取 20 条，查询速度直接崩掉。

根本原因在于这种分页方式依赖**物理偏移**，跳过的行越多，执行越慢。解决思路是避免使用大 offset，转而在**基于游标的分页**，也就是利用索引字段（通常是主键或有序列）做条件过滤。

比如你查订单列表，按创建时间排序，就可以记录上一页最后一条的时间戳和 ID，在下一页查询时加 where 条件：

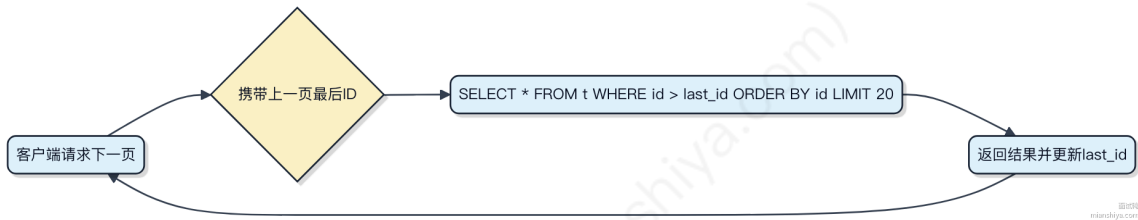
```
SELECT * FROM orders
WHERE id > 1000000
ORDER BY id
LIMIT 20;
```

这种方式能走主键索引，查询效率稳定在 $O(\log n)$ ，完全压根不经过前面那些无效扫描。

如果业务必须支持“跳转到第 N 页”，那就搞不定高性能了，只能提前算好每个分页的边界值，比如用延迟关联或者维护一个分页索引表，但成本高，一般推荐前端改成“加载更多”模式。

还有一个办法是结合业务场景用缓存，比如把热点数据预加载到 Redis，用 ZSET 按 score 排序后分页取，像 Feed 流系统里常见。

- 1) 核心是避免大 offset 的全范围扫描
- 2) 优先用 where + 索引字段替代 limit offset
- 3) 配合缓存或物化视图提升极端场景体验



详细描述一条 SQL 语句在 MySQL 中的执行过程。

一条 SQL 语句从发起到执行完成，MySQL 要走完一整套流程，涉及多个组件协作。

首先客户端发起请求，连接器先验证身份和权限，同时维持连接状态。长连接用得小心，内存占用会累积，8 小时空闲也会自动断开。

接着解析 SQL。**查询缓存**在这阶段起作用，但默认是关闭的，因为命中率低，8.0 直接移除了。然后词法和语法分析生成解析树，再通过预处理器校验字段、表名是否存在，权限是否够。

接下来进入优化阶段。优化器决定执行路径，比如多表 JOIN 的顺序、走哪个索引。它会基于成本模型选一个它认为“最快”的方案，生成**执行计划**。可以用 EXPLAIN 看这个计划。

执行时，优化器调用存储引擎接口，比如 InnoDB。如果是 SELECT，就通过索引找到数据行。InnoDB 的聚簇索引结构决定了主键查询最快，二级索引需要回表。更新操作还会写 redo log 和 undo log，保证崩溃恢复和事务回滚。

整个过程还可能涉及锁机制，比如行锁、间隙锁，在 RR 隔离级别下防止幻读。如果是事务，还得遵循两阶段提交，尤其是开启 binlog 的情况下，确保 crash-safe。



实际排查慢查询，一般从慢日志入手，结合 EXPLAIN 分析执行计划，看有没有全表扫描、索引失效等问题。

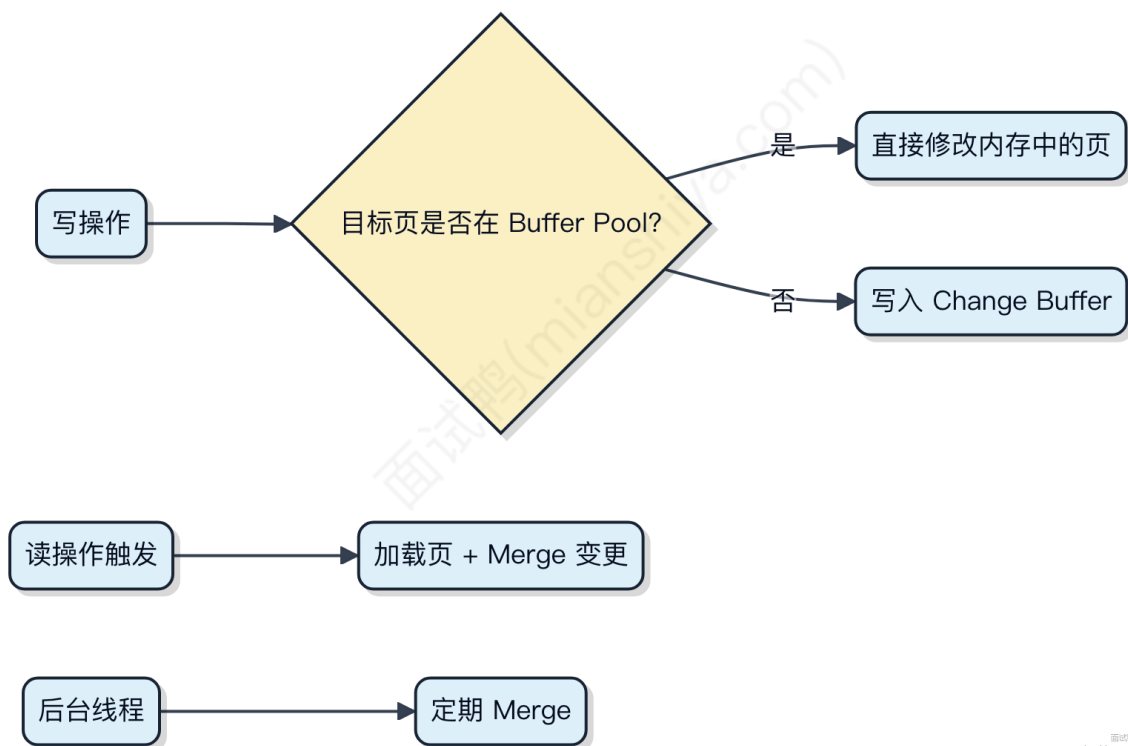
MySQL 的 Change Buffer 是什么？它有什么作用？

MySQL 的 Change Buffer 主要用来优化**非唯一二级索引**的写入性能。你想想，如果每次 INSERT、UPDATE 或 DELETE 都去更新磁盘上的二级索引页，那 IO 成本太高了，尤其是当这个页还没被加载到 buffer pool 的时候。

这时候 Change Buffer 就能派上用场。它先把对非唯一二级索引的修改记录下来，不急着刷回磁盘。等下次这个索引页被读取时，再把变更合并上去，这个过程叫 **merge**。这样一来，很多随机 IO 就变成了“懒加载”，大大减少了磁盘压力。

- 1) 只针对非唯一的二级索引生效，唯一索引不行，因为每次插入都得校验唯一性，必须把页读进来
- 2) Change Buffer 本身也是存在磁盘上的，但它是顺序写，成本低
- 3) 在 buffer pool 中的页被访问时触发 merge，也可能由后台线程定期合并

举个例子：你在用 MySQL 批量导入数据，表上有几个非唯一索引。如果没有 Change Buffer，每条记录都要去找对应的索引页，可能几千次 IO 变成几十次，性能差十倍都不止。



说白了，Change Buffer 是用“延迟更新”换 IO 效率，特别适合写多读少的负载。但如果你的业务频繁查询刚写过的索引，反而会频繁触发 merge，优势就没了。

从 MySQL 获取数据，是从磁盘读取的吗？（buffer pool）

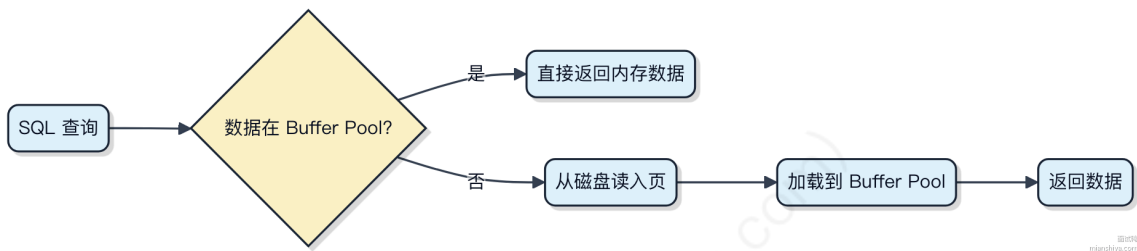
MySQL 查询数据，不一定每次都从磁盘读。关键在于 **Buffer Pool**，它是 InnoDB 存储引擎里的一块内存区域，用来缓存表数据和索引数据。

查询发起时，MySQL 会先查 Buffer Pool。如果数据页已经在内存里，直接返回，这叫一次“命中”。命中了就压根不经过磁盘，速度是内存访问级别，通常在几十到几百纳秒。如果没命中，才会去磁盘读取对应的数据页，加载进 Buffer Pool，再返回结果。这个过程慢得多，一次随机磁盘 IO 一般在 10ms 左右。

Buffer Pool 的大小很关键。生产环境通常会设置为物理内存的 70%~80%，比如机器有 32GB 内存，可能给 24GB 给 Buffer Pool。像阿里云 RDS 或者大型电商平台的数据库实例，基本都靠大 Buffer Pool 抗住高并发读请求。

写操作也绕不开它。更新数据其实是先改 Buffer Pool 中的页（变成脏页），然后由后台线程异步刷回磁盘。这种机制把随机 IO 变成了顺序 IO，大大减轻磁盘压力。

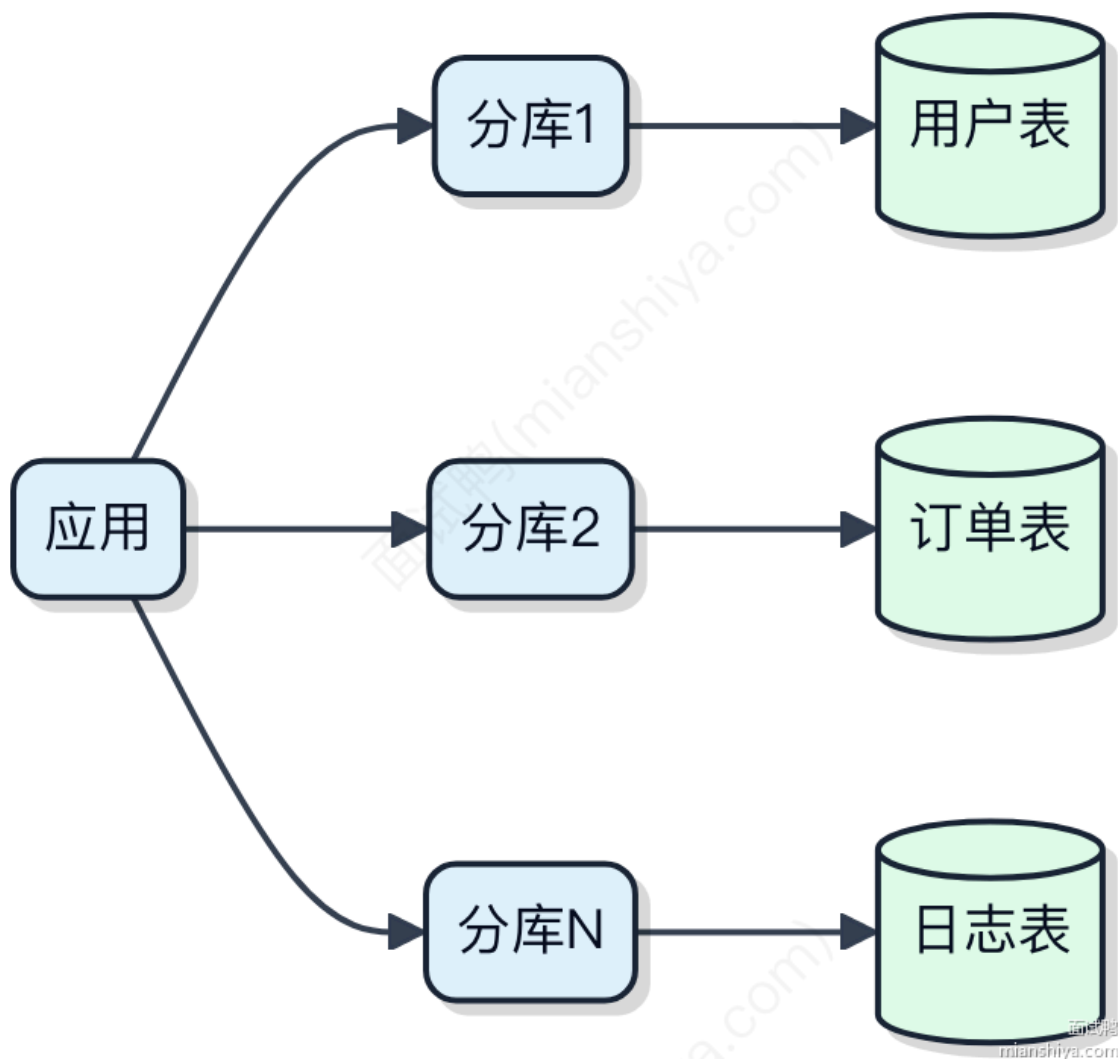
所以，MySQL 大部分时候是在内存里玩数据，Buffer Pool 就是那个让数据库快起来的“加速器”。



对数据库进行分库分表可能会引发哪些问题？

分库分表不是简单把数据拆开就完事了，一旦落地会带来一系列连锁问题，很多是设计阶段容易忽略的。

- 1) 跨库**事务**很难搞。原本一个事务搞定的操作，现在数据分布在不同库，本地事务压根不经过对方，得靠分布式事务方案，比如 Seata 这类框架，但性能损耗明显，复杂度也上去了。
- 2) 全局唯一主键不能再自增 ID。每个库的自增都从 1 开始，合起来就重复了。得引入分布式 ID 生成器，像雪花算法 (Snowflake)、美团的 Leaf，保证跨库 ID 不冲突。
- 3) 跨库查询和 **JOIN** 操作基本废了。你不能指望在两个物理数据库之间做高效的关联查询，结果通常是应用层手动拼接，或者通过宽表、冗余字段来规避，本质是用空间换时间。
- 4) 分页和聚合查询变慢。比如 `count(*)`、`order by`，可能要从多个库查出数据再合并排序，数据量一大延迟明显。有些场景得靠中间件（如 ShardingSphere）做归并处理，但仍有瓶颈。
- 5) 运维复杂度飙升。备份、扩容、DDL 变更都得跨多个库协调，一不留神就漏改。上线脚本必须带分片键，否则可能扫全表，直接把数据库打崩。



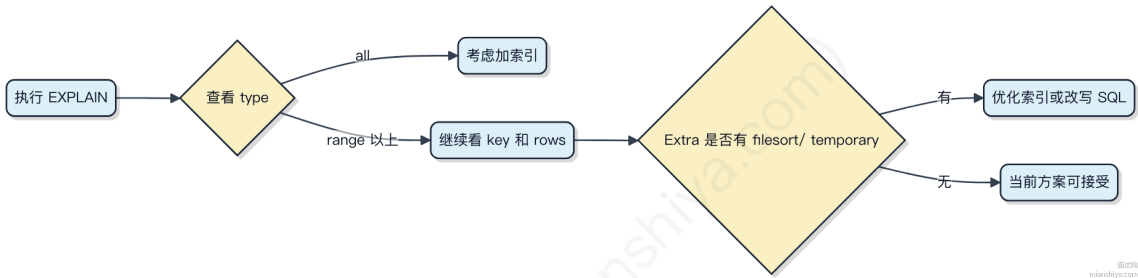
总的来说，单机数据库能扛住就别急着分，分了就得接受这些脏活累活。

如何使用 MySQL 的 EXPLAIN 语句进行查询分析？

EXPLAIN 是分析 SQL 执行计划的最直接工具，执行后返回的每行结果对应查询中的一张表，能看出 MySQL 是怎么执行这条语句的。

- 1) type 列看访问类型，从 system、const、eq_ref、ref 到 range、index、all，性能依次下降。一般优化至少要到 range 级别，all 意味着全表扫描，通常不可接受。
- 2) key 列显示实际走的索引，如果为 NULL 就得警惕了，大概率该建索引了。而 possible_keys 是可能用上的索引，不一定真用。
- 3) rows 列是预估扫描行数，越大越慢。比如查 10 条数据却要扫 10 万行，那肯定有问题。
- 4) Extra 最关键，出现 Using filesort 或 Using temporary 基本意味着需要优化。前者表示无法利用索引排序，后者是用了临时表，都是性能杀手。

举个例子，你用 ORDER BY 创建时间排序但没加索引，Extra 很可能就报 Using filesort。给创建时间字段加个 B+ 树索引，就能消除。

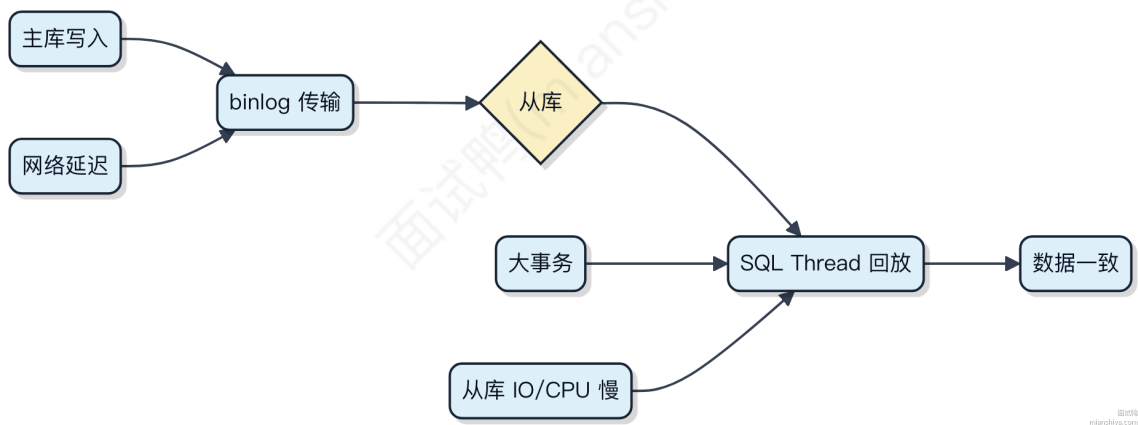


实际排查时结合 WHERE 条件、JOIN 顺序和索引设计一起看，像 **覆盖索引** 这种能让 Extra 显示 Using index 的情况，就是理想状态。

如何处理 MySQL 的主从同步延迟？

主从同步延迟本质是备库追不上主库的写入速度。常见于大事务、网络抖动或备库负载过高，得先定位瓶颈在哪一环。

- 1) 先看延迟来源。用 SHOW SLAVE STATUS 里的 Seconds_Behind_Master 只是参考，更要看 relay_log 回放进度。如果 SQL 线程卡住，可能是单线程回放大事务，比如一个 delete 删了几百万行，这时候主库瞬间完成，备库得一条条执行，压根不经过优化。
- 2) 解决方案分几类。如果是大事务导致，应用层尽量拆成小批量操作，别搞 10w 行的单 SQL。架构上可以考虑 **并行复制**，MySQL 5.7 之后支持按库并行，8.0 支持更细粒度的逻辑时钟并行，能明显提升回放速度。
- 3) 读写分离场景下，关键请求可以走主库。像用户注册完立刻查信息，这种强一致性需求，代码里就得标记走主库，别指望从库实时同步。TDDL、ShardingSphere 这类中间件都支持 hint 强制主库路由。
- 4) 监控也不能少。一旦延迟超过 30 秒就告警，结合日志判断是网络、IO 还是 SQL 回放问题。极端情况可以临时切主，但要考虑 GTID 断裂风险。



什么是 MySQL 的主从同步机制？它是如何实现的？

MySQL 的主从同步本质是数据变更事件的传递与回放。主库把每一笔写操作记录下来，从库拿到这些记录，在自己身上重新执行一遍，最终达到数据一致。

实现上靠三个关键步骤。主库开启 binlog，所有更改都会追加到这个日志文件里。从库启动一个 I/O 线程，连接主库，请求获取 binlog 里的事件，拉过来存到自己的 relay log（中继日志）里。接着，从库的 SQL 线程读取 relay log，按顺序重放事件，完成数据同步。



整个过程是异步的，从库不强制实时跟进。比如你用 MySQL 搭读写分离，像 ShardingSphere 或 MyCat 这类中间件就能把查询路由到从库，减轻主库压力。

有个坑得注意，主库崩了可能丢数据，因为事务提交后才写 binlog，还没来得及传出去就挂了。半同步复制能缓解这个问题，至少等一个从库确认收到才返回成功。

binlog 是核心载体，格式推荐用 row 模式，能精确记录每一行的变化，避免 statement 模式下因函数或触发器导致的

如何在 MySQL 中实现读写分离？

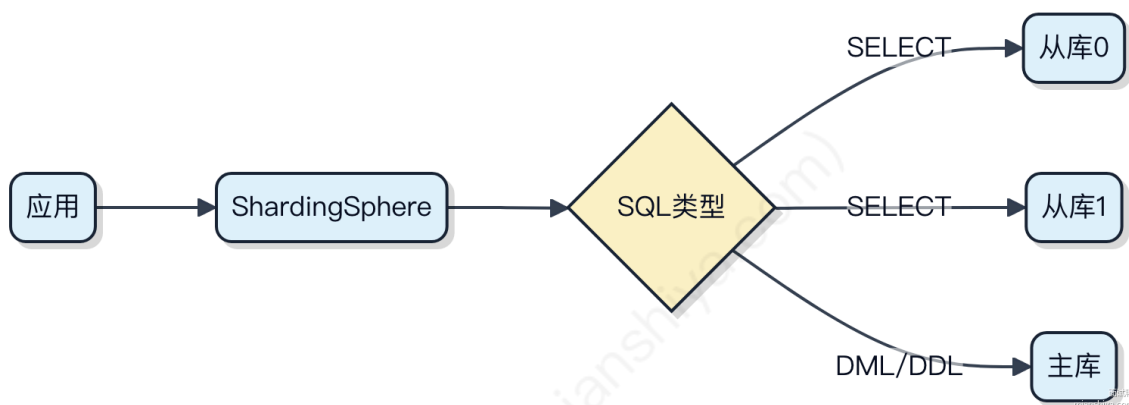
读写分离本质是把查询请求分发到只读副本，写操作集中在主库，靠的是 MySQL 的主从复制和中间层路由。

- 1) 主从复制是基础。主库（Master）把数据变更记录到 binlog，从库（Slave）拉取并重放，实现数据同步。延迟通常在毫秒到秒级，极端情况可能达到几秒。
- 2) 应用层或中间件做 SQL 路由。直接在代码里判断 SQL 类型太重，一般用 **ShardingSphere** 或 **MyCat** 这类中间件。它们解析 SQL，SELECT 自动打到从库，INSERT/UPDATE/DELETE 走主库。
- 3) 注意事务中的读必须走主库。比如一个事务里先更新用户余额，紧接着查，如果查被路由到延迟的从库，会读到旧值。这类场景要强制走主库，ShardingSphere 支持 hint 强制主库执行。
- 4) 从库可以挂多个，分摊读压力。但主库只有一个，写入瓶颈依然存在，所以读写分离解决不了写扩展问题。

简单配置示例（ShardingSphere YAML 片段）：

```
dataSources:
  master: ds-master
  slave_0: ds-slave-0
  slave_1: ds-slave-1
rules:
  - !READWRITE_SPLITTING
    dataSources:
      rw_ds:
        writeDataSourceName: master
        readDataSourceNames: [slave_0, slave_1]
```

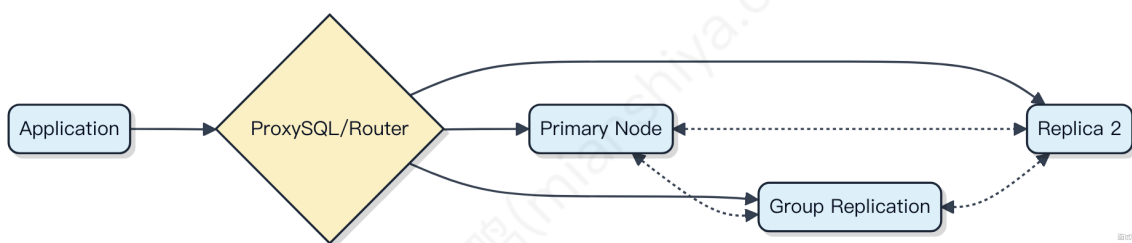
流量路由过程如下：



如何在 MySQL 中避免单点故障？

MySQL 要扛住高可用，单点故障必须干掉，核心思路是 **数据冗余 + 自动切换**。

- 1) 主从复制是最基础的一招，写操作走主库，读请求分摊到多个从库。主库挂了，人工或脚本把某个从库提拔成新主库。但这个过程有空窗期，没法完全避免数据丢失，适合容忍短时不可用的场景。
- 2) 想要自动兜底，得上 **高可用架构**。比如用 MHA (Master High Availability)，它能监控主库状态，一旦发现主库失联，自动选取一个数据最完整的从库进行切换，并更新其他从库的复制指向。整个过程几十秒内完成，比人肉快多了。
- 3) 更进一步，可以用 MySQL Group Replication 或基于它的 InnoDB Cluster。这玩意儿底层靠 Paxos 类协议同步数据，保证多数派确认才提交，任何单点宕机都不影响整体服务。配合 Router 组件，应用几乎无感。
- 4) 别忘了外部依赖也要去单点。比如用 Keepalived 搞虚拟 IP，主库对外暴露的是 VIP，切换时把 VIP 漂到新主库；或者让应用连中间件，像 MySQL Router、ProxySQL，它们自己维护后端实例列表，自动路由流量。



说白了，主从复制解决不了自动 failover 的问题，要真正在意 SLA，得往上走到 MHA 或原生 Group Replication 这种带协调能力的方案。

MySQL 中如何进行 SQL 调优？

SQL 调优不是上来就改语句，先知道慢在哪。第一步永远是开慢查询日志，把执行时间超过 1 秒的 SQL 捞出来，这个阈值可以根据业务调整，比如线上核心接口可能得设成 100ms。

找到慢 SQL 后，用 **explain** 看执行计划。重点关注 type 字段，如果是 ALL 就是全表扫描，index 是全索引扫描，最好能做到 range 或 ref。Extra 里出现 Using filesort 或 Using temporary 基本就有性能问题，说明排序或分组没走好索引。

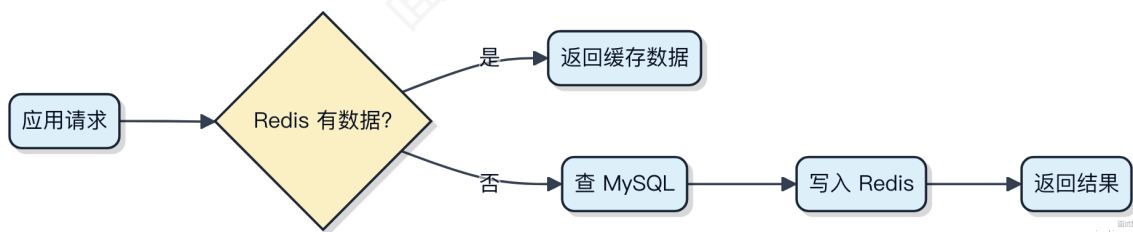
索引设计要结合查询条件。比如经常按 user_id 和 create_time 查订单，那 (user_id, create_time) 的联合索引就很合适。记住最左前缀原则，查询时如果跳过 user_id 直接查 create_time，这个联合索引就用不上。

```
-- 比如这张订单表
SELECT * FROM orders WHERE user_id = 123 AND create_time > '2023-01-01';
-- 建立复合索引
CREATE INDEX idx_user_time ON orders(user_id, create_time);
```

别忘了索引也有成本，写多读少的表加太多索引会拖慢插入更新。大字段比如 text 类型别放索引里，可以用前缀索引或者单独拎出去。

还有些场景需要改架构。单表数据上亿就得考虑分库分表，ShardingSphere 或者 MyCat 可以帮忙。但分了之后跨库 join 和事务就是难题，得从业务设计上规避。

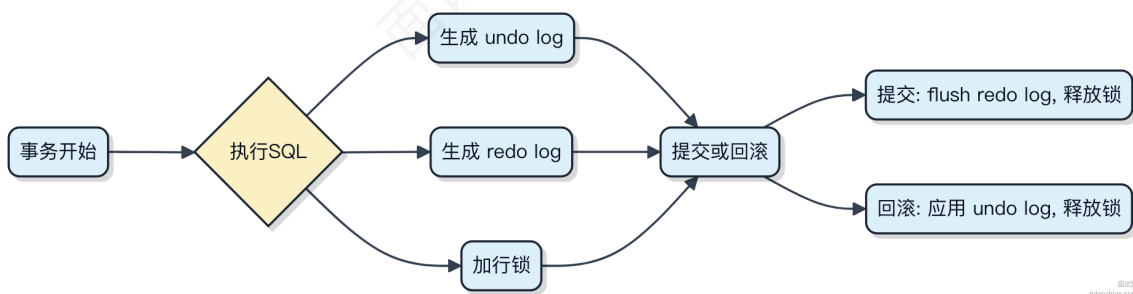
最后提一嘴缓存。热点数据像商品信息，可以放 Redis，避免频繁打到数据库。但要注意缓存穿透、击穿这些坑，合理设置过期时间和空值。



MySQL 是如何实现事务的?

MySQL 的事务能力由存储引擎层实现，最典型的是 InnoDB 引擎。它通过 redo log、undo log 和 锁机制 三驾马车协同工作来保证 ACID。

- 1) redo log 确保持久性。当数据更新时，InnoDB 先写日志再改内存中的数据页，后续刷盘异步进行。即使宕机重启，也能靠 redo log 把未落盘的变更重放一遍。这个过程叫 crash recovery。
- 2) undo log 支持原子性和一致性。每个修改操作都会生成反向操作的日志。如果事务回滚，就用 undo log 恢复到修改前的状态。同时 MVCC（多版本并发控制）也依赖 undo log 构建历史版本，让不同事务看到一致的数据快照。
- 3) 锁机制保障隔离性。InnoDB 在行级别加锁，比如读用共享锁，写用排他锁。配合 next-key lock（记录锁 + 间隙锁），能防止幻读。不同的事务隔离级别会调整锁策略和可见性规则。

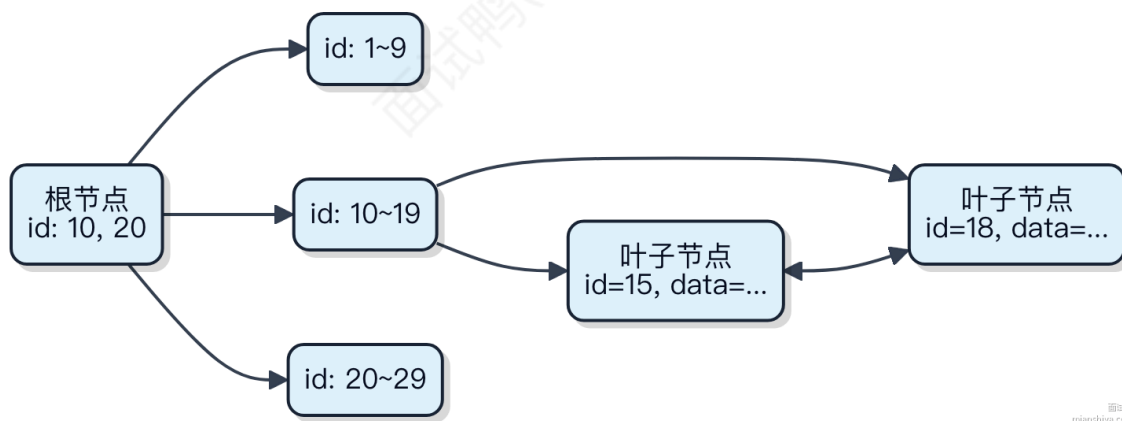


实际应用中，像支付宝转账这种场景就必须依赖这套机制，确保要么全部成功，要么全部回退。如果只靠内存操作或者普通文件写入，根本扛不住并发和故障。

请详细描述 MySQL 的 B+ 树中查询数据的全过程

MySQL 的 InnoDB 引擎用 B+ 树组织主键索引，数据查询就是从根节点一路往下找叶子节点的过程。非叶子节点存的是索引键和指向子节点的指针，只有叶子节点才真正存数据，并且叶子之间用双向链表串起来，方便范围查询。

- 1) 从根节点开始，根据查询条件比如 `WHERE id = 15`，在当前节点的键值中做二分查找，找到目标值所属的区间，决定下一步走哪个子节点。
- 2) 逐层下探，每次 IO 读一个数据页（默认 16KB），直到抵达叶子节点。这个过程一般不超过 3~4 层，也就是说多数查询 3 次磁盘 IO 就能定位到数据。
- 3) 在对应的叶子节点页内，再通过二分加遍历的方式找到具体的记录。如果查的是普通索引，可能还要回表一次主键索引拿完整数据。



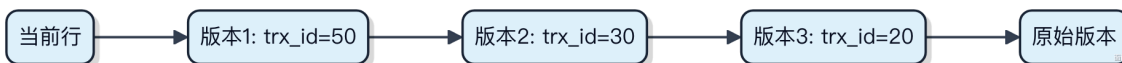
注意，B+ 树所有查询都必须走到叶子节点才算结束，不像 B 树可能中途就命中。这也是它更适合数据库的原因——查询路径稳定，而且叶子连成链表后，像 `BETWEEN 15 AND 25` 这种范围扫描效率很高。聚簇索引下，主键有序插入能减少页分裂，随机写入则容易产生碎片。

MySQL 中的 MVCC 是什么？

MVCC，全称多版本并发控制，是 MySQL InnoDB 存储引擎实现高并发读的核心机制。它让读操作不用加锁也能看到一致的数据视图，读写之间不互相阻塞。

InnoDB 通过三样东西配合实现 MVCC：隐藏的 `DB_TRX_ID`（事务 ID）、`DB_ROLL_PTR`（回滚指针），再加上 **undo log**。每次修改行记录时，旧版本数据会被写进 undo log，新行指向它，形成一个版本链。

事务启动时，InnoDB 会拍一个活跃事务 ID 快照，构建 Read View。这个视图决定了你能看到哪个版本的数据。比如当前事务看不到还没提交的修改，也看不到在你之后启动的事务做的变更。



像可重复读（RR）隔离级别下，Repeatable Read 靠的就是事务开始时生成的那个 Read View 贯穿始终，中间其他事务改了数据并提交，你也看不见，保证了多次读结果一致。

但注意，MVCC 只作用于普通 `SELECT` 这种快照读。如果是 `SELECT ... FOR UPDATE` 或 `LOCK IN SHARE MODE`，那就是当前读，直接查最新已提交数据，得加锁。

更新操作也走当前读，所以 RR 级别能防幻读，靠的不是 MVCC，是 next-key lock。简单说，MVCC 是用来“扛住”大量并发读的脏活累活，让数据库在高并发下依然能快速响应查询。

如果 MySQL 中没有 MVCC，会有什么影响？

没有 MVCC，MySQL 的并发性能会直接垮掉。最直观的表现就是读写互相堵着，一个事务在改数据，其他所有事务的查询都得等着，响应时间从毫秒级飙升到几秒甚至超时。

- 1) 在更新频繁的场景下，比如电商扣库存，大量 SELECT 请求会被 UPDATE 锁住，用户体验就是页面卡死。InnoDB 默认的可重复读隔离级别也实现不了，你看到的数据可能正在被别人改，结果前后两次查出来不一样。
- 2) 没有 MVCC 就只能靠纯锁机制实现隔离，像快照读这种能力压根不经过锁，靠的是 **undo log** 拿历史版本数据。有了 MVCC，普通 SELECT 不加锁，读到的是事务开始时的快照，写操作才加行锁。这才让读写可以并行。
- 3) 像 Redis 这种内存数据库没 MVCC 问题不大，因为它操作是原子的、执行快。但 MySQL 是持久化存储，IO 成本高，一旦读被锁住，磁盘利用率上不去，QPS 很容易卡在几百几千。

代码上看，MVCC 的关键在于 `DB_ROLL_PTR` 和 `DB_TRX_ID` 这两个隐藏字段。每次修改数据，旧值被写进 undo log，指针串起来形成版本链。查询时根据自己的事务 ID 和活跃事务列表，决定该“看见”哪个版本。

说白了，MVCC 是用空间换时间，把并发控制从“排队等锁”变成“各看各的”，这才是 InnoDB 能扛住高并发读写的底牌。

本资源来自面试鸭：<https://www.mianshiya.com>

推荐更多免费学编程资源：

1. 编程导航学习网站：[学编程、做项目、拿 Offer!](#)
2. 企业高频面试题库：[开始刷题，面试遇原题!](#)
3. 精选简历模板大全：[1 分钟搞定简历!](#)
4. AI 资源导航网站：[获取最新 AI 黑科技!](#)
5. 1 对 1 模拟面试：[随时随地提升面试能力](#)